

Análisis de la salud del sueño en relación al estilo de vida

Santiago López y Aleix Pagès

2025-10-24

Contents

Máster MADI	1
1. ¿Existen diferencias en la duración media del sueño entre hombres y mujeres?	27
2. ¿La calidad del sueño difiere según la categoría de IMC?	38
3. ¿Qué variables explican mejor la calidad del sueño?	46
4. ¿Tener una alteración del sueño influye significativamente en la calidad del sueño?	52
5. ¿Podemos predecir si una persona sufre insomnio o apnea?	56
6. ¿Existen patrones latentes que agrupan a los individuos?(Tambien he eliminado quality of sleep)	61

Máster MADI

Importamos todas las librerías que vamos a necesitar.

Descripción del DATASET El DataSet, llamado Sleep Health and Lifestyle Dataset, que se ha usado en este trabajo se ha obtenido de Kaggle. En este, se da información sobre datos de salud del sueño y estilo de vida de los pacientes.

El Dataset consta de 400 filas y 13 columnas (variables) y, abarca una amplia variedad de aspectos relacionadas con el sueño y los hábitos de vida diarios.

Las variables del DataSet son:

1. Person ID

- **Tipo:** Numérica entera (`int64`)
 - **Descripción:** Identificador único de cada individuo.
 - **Observación:** Variable usada solo para identificación, no aporta contenido analítico.
-

2. Gender

- **Tipo:** Categórica (`object`)
 - **Descripción:** Sexo biológico del participante.
 - **Valores posibles:** Male, Female.
-

3. Age

- **Tipo:** Numérica entera (`int64`)
 - **Descripción:** Edad del individuo en años.
-

4. Occupation (Profesión)

- **Tipo:** Categórica (`object`)
 - **Descripción:** Profesión, como por ejemplo: *Doctor, Software Engineer, Sales Representative*.
 - **Uso:** Analizar patrones de sueño o estrés según la profesión.
-

5. Sleep Duration

- **Tipo:** Numérica decimal (`float64`)
 - **Unidad:** Horas por día.
 - **Descripción:** Tiempo total de sueño diario.
-

6. Quality of Sleep

- **Tipo:** Numérica entera (`int64`)
 - **Escala:** 1 a 10.
 - **Descripción:** Calidad subjetiva del sueño percibida por el individuo.
-

7. Physical Activity Level

- **Tipo:** Numérica entera (`int64`)
 - **Unidad:** Minutos por día.
 - **Descripción:** Minutos de actividad física diaria.
-

8. Stress Level

- **Tipo:** Numérica entera (`int64`)
 - **Escala:** 1 a 10.
 - **Descripción:** Nivel de estrés percibido.
-

9. BMI Category

- **Tipo:** Categórica (`object`)
 - **Valores posibles:** Underweight, Normal, Overweight, Obese.
 - **Descripción:** Clasificación del Índice de Masa Corporal.
-

10. Blood Pressure

- **Tipo:** Categórica (`object`)
 - **Formato:** "Sistólica/Diastólica" (ejemplo: "126/83").
 - **Descripción:** Presión arterial en mmHg.
-

11. Heart Rate

- **Tipo:** Numérica entera (`int64`)
 - **Unidad:** Latidos por minuto (bpm).
 - **Descripción:** Frecuencia cardíaca promedio.
-

12. Daily Steps

- **Tipo:** Numérica entera (`int64`)
 - **Unidad:** Pasos por día.
 - **Descripción:** Número total de pasos diarios del individuo.
-

13. Sleep Disorder

- **Tipo:** Categórica (`object`)
- **Valores posibles:** "None", "Insomnia", "Sleep Apnea".
- **Descripción:** Indica si el individuo presenta trastornos del sueño.

Pasos del estudio Carga del DataSet y análisis exploratorio inicial.

1. Estudio sobre la duración media del sueño entre hombres y mujeres.
2. Asociación entre el índice de masa corporal (BMI) y la calidad del sueño.
3. Estudio de variables que predigan mejor la calidad del sueño.
4. Relación entre la calidad del sueño y las alteraciones del sueño.
5. Estudio sobre la predicción de trastornos del sueño.
6. Análisis de patrones en los individuos del DataSet.

Carga y Preparación de los datos Primero de todo cargamos los datos y realizamos una exploración inicial.

```
# Cargar el dataset.
# La ruta es relativa a la raíz del repositorio (ver opts_knit en el bloque setup),
# de modo que el documento se teje igual desde RStudio que desde la línea de comandos.
df_1 <- read.csv(file.path("data", "Sleep_health_and_lifestyle_dataset.csv"),
                  stringsAsFactors = FALSE)
# Exploración inicial
cat("Dimensiones del dataset:\n")
```

Dimensiones del dataset:

```
print(dim(df_1))
```

[1] 374 13

```
cat("Estructura del dataset:\n")
```

Estructura del dataset:

```
str(df_1)
```

```
## 'data.frame': 374 obs. of 13 variables:
## $ Person.ID : int 1 2 3 4 5 6 7 8 9 10 ...
## $ Gender : chr "Male" "Male" "Male" "Male" ...
## $ Age : int 27 28 28 28 28 28 29 29 29 29 ...
## $ Occupation : chr "Software Engineer" "Doctor" "Doctor" "Sales Representative" ...
## $ Sleep.Duration : num 6.1 6.2 6.2 5.9 5.9 5.9 6.3 7.8 7.8 7.8 ...
## $ Quality.of.Sleep : int 6 6 6 4 4 4 6 7 7 7 ...
## $ Physical.Activity.Level: int 42 60 60 30 30 30 40 75 75 75 ...
## $ Stress.Level : int 6 8 8 8 8 8 7 6 6 6 ...
## $ BMI.Category : chr "Overweight" "Normal" "Normal" "Obese" ...
## $ Blood.Pressure : chr "126/83" "125/80" "125/80" "140/90" ...
## $ Heart.Rate : int 77 75 75 85 85 85 82 70 70 70 ...
## $ Daily.Steps : int 4200 10000 10000 3000 3000 3000 3500 8000 8000 8000 ...
## $ Sleep.Disorder : chr "None" "None" "None" "Sleep Apnea" ...
```

```
cat("Primeras filas:\n")
```

Primeras filas:

```
head(df_1)
```

```
## Person.ID Gender Age Occupation Sleep.Duration Quality.of.Sleep
## 1 1 Male 27 Software Engineer 6.1 6
## 2 2 Male 28 Doctor 6.2 6
## 3 3 Male 28 Doctor 6.2 6
## 4 4 Male 28 Sales Representative 5.9 4
## 5 5 Male 28 Sales Representative 5.9 4
```

```
## 6      6   Male  28   Software Engineer      5.9      4
##   Physical.Activity.Level Stress.Level BMI.Category Blood.Pressure Heart.Rate
## 1              42              6   Overweight      126/83      77
## 2              60              8     Normal      125/80      75
## 3              60              8     Normal      125/80      75
## 4              30              8      Obese      140/90      85
## 5              30              8      Obese      140/90      85
## 6              30              8      Obese      140/90      85
##   Daily.Steps Sleep.Disorder
## 1         4200          None
## 2        10000          None
## 3        10000          None
## 4         3000   Sleep Apnea
## 5         3000   Sleep Apnea
## 6         3000    Insomnia
```

```
cat("Resumen estadístico:\n")
```

```
## Resumen estadístico:
```

```
summary(df_1)
```

```
##   Person.ID      Gender      Age      Occupation
## Min.   : 1.00  Length:374  Min.   :27.00  Length:374
## 1st Qu.: 94.25  Class :character 1st Qu.:35.25  Class :character
## Median :187.50  Mode  :character Median :43.00  Mode  :character
## Mean   :187.50                      Mean   :42.18
## 3rd Qu.:280.75                      3rd Qu.:50.00
## Max.   :374.00                      Max.   :59.00
## Sleep.Duration Quality.of.Sleep Physical.Activity.Level Stress.Level
## Min.   :5.800  Min.   :4.000  Min.   :30.00  Min.   :3.000
## 1st Qu.:6.400  1st Qu.:6.000  1st Qu.:45.00  1st Qu.:4.000
## Median :7.200  Median :7.000  Median :60.00  Median :5.000
## Mean   :7.132  Mean   :7.313  Mean   :59.17  Mean   :5.385
## 3rd Qu.:7.800  3rd Qu.:8.000  3rd Qu.:75.00  3rd Qu.:7.000
## Max.   :8.500  Max.   :9.000  Max.   :90.00  Max.   :8.000
## BMI.Category      Blood.Pressure      Heart.Rate      Daily.Steps
## Length:374      Length:374      Min.   :65.00  Min.   : 3000
## Class :character  Class :character 1st Qu.:68.00  1st Qu.: 5600
## Mode  :character  Mode  :character Median :70.00  Median : 7000
##                      Mean   :70.17  Mean   : 6817
##                      3rd Qu.:72.00  3rd Qu.: 8000
##                      Max.   :86.00  Max.   :10000
## Sleep.Disorder
## Length:374
## Class :character
## Mode  :character
##
##
##
```

```
cat("\nInterpretación del resumen:\n")
```

```
##
## Interpretación del resumen:
```

```
cat("- Edad: rango entre", min(df_1$Age, na.rm = TRUE), "y", max(df_1$Age, na.rm = TRUE),
    "años, media de", round(mean(df_1$Age, na.rm = TRUE), 1), "años.\n")
```

```
## - Edad: rango entre 27 y 59 años, media de 42.2 años.
```

```
cat("- Duración del sueño: media de", round(mean(df_1$Sleep.Duration, na.rm = TRUE), 2),
    "horas, con variación entre", round(min(df_1$Sleep.Duration, na.rm = TRUE), 2),
    "y", round(max(df_1$Sleep.Duration, na.rm = TRUE), 2), "horas.\n")
```

```
## - Duración del sueño: media de 7.13 horas, con variación entre 5.8 y 8.5 horas.
```

```
cat("- Nivel de estrés: escala 1-10, media de",
    round(mean(df_1$Stress.Level, na.rm = TRUE), 2), ".\n")
```

```
## - Nivel de estrés: escala 1-10, media de 5.39 .
```

```
cat("Valores faltantes:\n")
```

```
## Valores faltantes:
```

```
colSums(is.na(df_1))
```

```
##          Person.ID          Gender          Age
##              0              0              0
##      Occupation      Sleep.Duration      Quality.of.Sleep
##              0              0              0
## Physical.Activity.Level      Stress.Level      BMI.Category
##              0              0              0
##      Blood.Pressure      Heart.Rate      Daily.Steps
##              0              0              0
##      Sleep.Disorder
##              0
```

Como se puede observar, el conjunto de datos no presenta valores faltantes, por lo que no ha sido necesario eliminar filas o columnas ni aplicar ningún tipo de imputación. Esto permite mantener los datos tal y como fueron obtenidos, evitando pasos extra de limpieza y posibles sesgos por modificaciones innecesarias.

```
# Limpieza de nombres de columnas (por si hay espacios).
colnames(df_1) <- make.names(colnames(df_1))
```

Calidad de los datos

Duplicados

```
# Contar filas totales vs filas únicas
filas_totales <- nrow(df_1)
filas_unicas <- nrow(dplyr::distinct(df_1))
num_duplicados <- filas_totales - filas_unicas

cat(paste("Total de registros:", filas_totales, "\n"))
```

```
## Total de registros: 374
```

```
cat(paste("Registros únicos:", filas_unicas, "\n"))
```

```
## Registros únicos: 374
```

```
cat(paste("Registros duplicados encontrados:", num_duplicados, "\n"))
```

```
## Registros duplicados encontrados: 0
```

```
# Verificar también duplicados por Person.ID
if ("Person.ID" %in% names(df_1)) {
  ids_unicos <- length(unique(df_1$Person.ID))
  ids_duplicados <- filas_totales - ids_unicos
  cat(paste("IDs únicos:", ids_unicos, "\n"))
  cat(paste("IDs duplicados:", ids_duplicados, "\n"))
}
```

```
## IDs únicos: 374
```

```
## IDs duplicados: 0
```

```
if (num_duplicados == 0) {
  cat("\n No existen filas duplicadas en el dataset.\n")
  cat("Interpretación: Cada registro representa un individuo único, garantizando\n")
  cat("la independencia de las observaciones.\n")
} else {
  cat("\n Se detectaron", num_duplicados, "duplicados.\n")
  cat("Recomendación: Considerar eliminación si corresponden a errores de registro.\n")

  # Mostrar ejemplos de duplicados usando dplyr explícitamente
  filas_duplicadas <- df_1 %>%
    dplyr::group_by_all() %>%
    dplyr::filter(dplyr::n() > 1) %>%
    dplyr::ungroup() %>%
    dplyr::arrange(dplyr::across(everything()))

  if (nrow(filas_duplicadas) > 0) {
    cat("\nPrimeras filas duplicadas encontradas:\n")
    print(head(filas_duplicadas, 6))
  }
}
```

```
##
```

```
## No existen filas duplicadas en el dataset.
```

```
## Interpretación: Cada registro representa un individuo único, garantizando
```

```
## la independencia de las observaciones.
```

Variabilidad de las columnas

```
variabilidad <- df_1 %>%  
  summarise(across(where(is.numeric), ~ n_distinct(.)))  
  
cat("Variables numéricas con baja variabilidad (< 5 valores únicos):\n")
```

```
## Variables numéricas con baja variabilidad (< 5 valores únicos):
```

```
bajas_var <- variabilidad[, variabilidad < 5]  
if (length(bajas_var) > 0) {  
  print(bajas_var)  
  cat("\nVariables con pocos valores únicos pueden ser tratadas como\n")  
  cat("categóricas ordinales en análisis posteriores.\n")  
} else {  
  cat("Todas las variables numéricas presentan variabilidad adecuada.\n")  
}
```

```
## Todas las variables numéricas presentan variabilidad adecuada.
```

Outliers

```
variables_numericas <- df_1 %>% select(where(is.numeric), -Person.ID)  
  
cat("Aplicando método IQR (Rango Intercuartílico) para detectar outliers:\n\n")
```

```
## Aplicando método IQR (Rango Intercuartílico) para detectar outliers:
```

```
outliers_resumen <- data.frame()  
for (col in names(variables_numericas)) {  
  Q1 <- quantile(variables_numericas[[col]], 0.25, na.rm = TRUE)  
  Q3 <- quantile(variables_numericas[[col]], 0.75, na.rm = TRUE)  
  IQR_val <- Q3 - Q1  
  limite_inf <- Q1 - 1.5 * IQR_val  
  limite_sup <- Q3 + 1.5 * IQR_val  
  
  outliers <- sum(variables_numericas[[col]] < limite_inf |  
                 variables_numericas[[col]] > limite_sup, na.rm = TRUE)  
  
  outliers_resumen <- rbind(outliers_resumen,  
                           data.frame(Variable = col,  
                                       Outliers = outliers,  
                                       Porcentaje = round(100 * outliers / nrow(df_1), 2)))  
}  
  
print(outliers_resumen)
```

```
##           Variable Outliers Porcentaje  
## 1           Age         0         0.00  
## 2 Sleep.Duration         0         0.00  
## 3 Quality.of.Sleep         0         0.00
```



```
## 4 Physical.Activity.Level      0      0.00
## 5           Stress.Level      0      0.00
## 6           Heart.Rate     15      4.01
## 7           Daily.Steps      0      0.00
```

```
cat("\nInterpretación: Los outliers pueden representar casos extremos legítimos\n")
```

```
##
```

```
## Interpretación: Los outliers pueden representar casos extremos legítimos
```

```
cat("(ej.: personas con hábitos de sueño inusuales) o errores de registro.\n")
```

```
## (ej.: personas con hábitos de sueño inusuales) o errores de registro.
```

```
cat("Se recomienda análisis caso por caso antes de decidir su tratamiento.\n")
```

```
## Se recomienda análisis caso por caso antes de decidir su tratamiento.
```

```
# Visualización de outliers
# Paleta accesible para daltonismo
color_box <- "#0173B2"      # Azul brillante para cajas
color_outlier <- "#DE8F05"  # Naranja para outliers (visible para daltónicos)
color_background <- "#F0F0F0" # Gris claro para strips

# Gráfica en estilo Nature
variables_numericas %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = Variable, y = Valor)) +
  # Boxplot con colores accesibles
  geom_boxplot(fill = color_box,
               alpha = 0.6,
               color = "black",
               linewidth = 0.5,
               outlier.colour = color_outlier,
               outlier.size = 2.5,
               outlier.shape = 16,
               outlier.alpha = 0.8) +

  # Facetas
  facet_wrap(~ Variable, scales = "free", ncol = 3) +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    # Panel y fondo
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major.y = element_line(color = "#EOEOEO", linewidth = 0.3),
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

    # Títulos y texto
    plot.title = element_text(size = 14, face = "bold", hjust = 0,
```

```

        margin = margin(b = 5)),
plot.subtitle = element_text(size = 10, hjust = 0,
                             color = "grey30",
                             margin = margin(b = 10)),

# Etiquetas de facetas
strip.background = element_rect(fill = color_background,
                                color = "black",
                                linewidth = 0.5),
strip.text = element_text(size = 9, face = "bold",
                          margin = margin(3, 3, 3, 3)),

# Ejes
axis.title = element_text(size = 10, face = "bold"),
axis.title.x = element_blank(),
axis.text.y = element_text(size = 8, color = "black"),
axis.text.x = element_blank(),
axis.line = element_line(color = "black", linewidth = 0.5),
axis.ticks.y = element_line(color = "black", linewidth = 0.5),
axis.ticks.x = element_blank(),

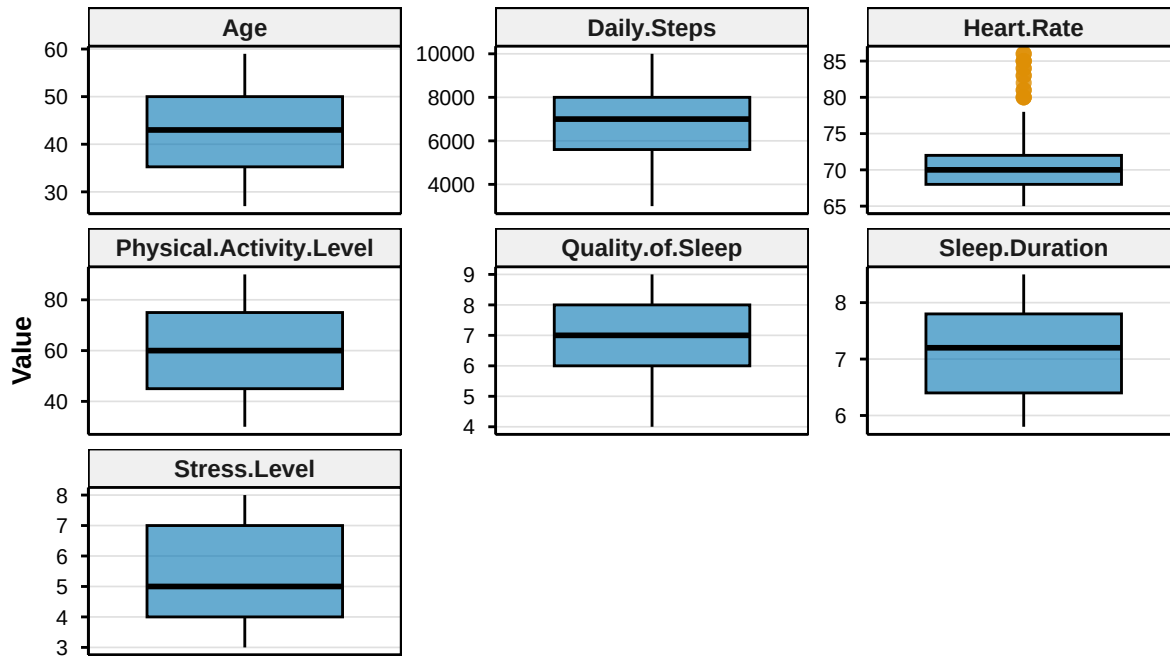
# Sin leyenda
legend.position = "none",

# Márgenes
plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Outlier Detection in Numerical Variables",
  subtitle = "Box plots with IQR-based outlier identification (orange points)",
  y = "Value",
  x = "",
  caption = "Outliers: Q1 - 1.5×IQR or Q3 + 1.5×IQR | Colorblind-friendly palette"
)

```

Outlier Detection in Numerical Variables

Box plots with IQR-based outlier identification (orange points)



Outliers: $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ | Colorblind-friendly palette

Distribución de las variables numéricas

```
# Paleta accesible para daltonismo (siguiendo guías Nature/Science)
color_histogram <- "#0173B2" # Azul brillante
color_density <- "#DE8F05" # Naranja/amarillo
color_background <- "#F0F0F0" # Gris muy claro

# Gráfica
variables_numericas %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = Valor)) +
  # Histograma con borde definido
  geom_histogram(aes(y = after_stat(density)),
    bins = 30,
    fill = color_histogram,
    alpha = 0.6,
    color = "white",
    linewidth = 0.3) +
  # Curva de densidad más gruesa y visible
  geom_density(color = color_density,
    linewidth = 1.2) +
  # Facetas con diseño limpio
  facet_wrap(~ Variable, scales = "free", ncol = 3) +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
```

```

# Panel y fondo
panel.background = element_rect(fill = "white", color = NA),
panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
panel.grid.minor = element_blank(),
panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

# Títulos y texto
plot.title = element_text(size = 14, face = "bold", hjust = 0,
                           margin = margin(b = 5)),
plot.subtitle = element_text(size = 10, hjust = 0,
                              color = "grey30",
                              margin = margin(b = 10)),

# Etiquetas de facetas
strip.background = element_rect(fill = color_background,
                                 color = "black",
                                 linewidth = 0.5),
strip.text = element_text(size = 9, face = "bold",
                           margin = margin(3, 3, 3, 3)),

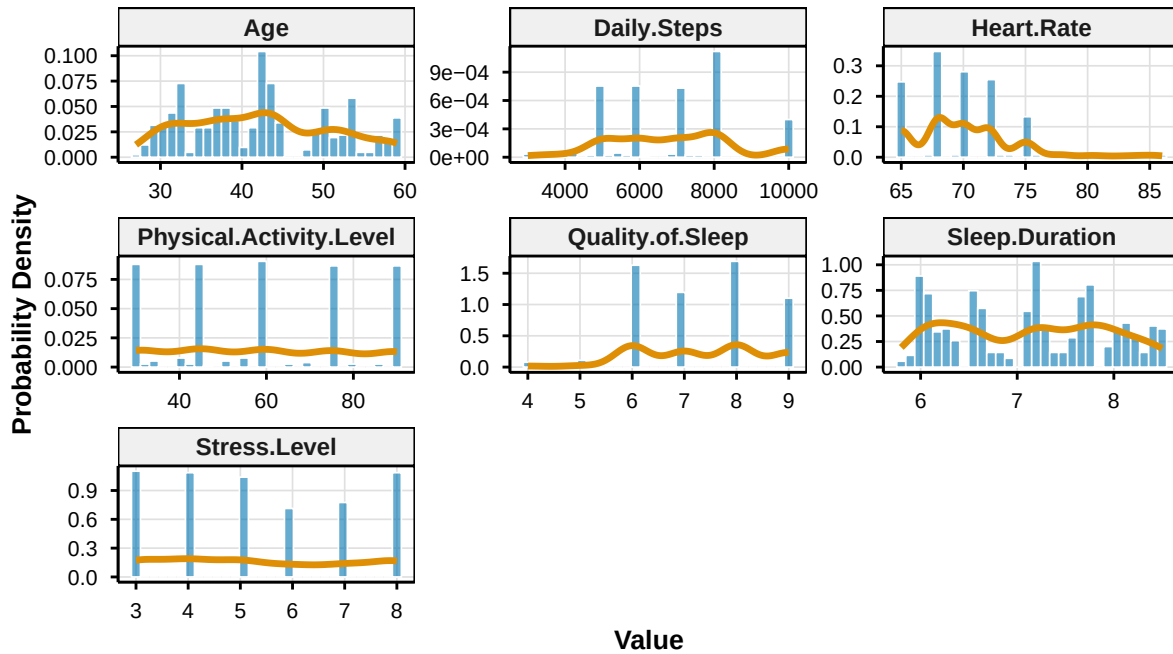
# Ejes
axis.title = element_text(size = 10, face = "bold"),
axis.text = element_text(size = 8, color = "black"),
axis.line = element_line(color = "black", linewidth = 0.5),
axis.ticks = element_line(color = "black", linewidth = 0.5),

# Márgenes
plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Distribution of Numerical Variables",
  subtitle = "Histogram (blue) with kernel density estimation (orange)",
  y = "Probability Density",
  x = "Value",
  caption = "Colorblind-friendly palette | Bins = 30"
)

```

Distribution of Numerical Variables

Histogram (blue) with kernel density estimation (orange)



Colorblind-friendly palette | Bins = 30

```
cat("\nInterpretación: Los histogramas revelan la forma de las distribuciones.\n")
```

```
##
```

```
## Interpretación: Los histogramas revelan la forma de las distribuciones.
```

```
cat("- Distribuciones simétricas sugieren normalidad (aplicable para test paramétricos).\n")
```

```
## - Distribuciones simétricas sugieren normalidad (aplicable para test paramétricos).
```

```
cat("- Distribuciones asimétricas pueden requerir transformaciones (log, sqrt).\n")
```

```
## - Distribuciones asimétricas pueden requerir transformaciones (log, sqrt).
```

Distribución de las variables categóricas

```
# Gender
tabla_gender <- table(df_1$Gender)
cat("\nDistribución por Género:\n")
```

```
##
```

```
## Distribución por Género:
```

```
print(tabla_gender)
```

```
##  
## Female    Male  
##      185    189
```

```
cat(paste("Proporción:", round(prop.table(tabla_gender) * 100, 1), "%\n"))
```

```
## Proporción: 49.5 %  
## Proporción: 50.5 %
```

```
library(ggplot2)  
library(dplyr)
```

```
# Paleta accesible para daltonismo - colores neutros y distinguibles
```

```
gender_colors <- c(  
  "Male"   = "#0173B2", # Azul  
  "Female" = "#DE8F05"  # Naranja  
)
```

```
# Gráfica
```

```
df_1 %>%  
  count(Gender) %>%  
  ggplot(aes(x = Gender, y = n, fill = Gender)) +  
  # Barras con borde negro definido  
  geom_col(alpha = 0.85,  
    color = "black",  
    linewidth = 0.5,  
    width = 0.7) +  
  # Etiquetas de valores mejoradas  
  geom_text(aes(label = scales::comma(n)),  
    vjust = -0.5,  
    size = 5,  
    fontface = "bold",  
    color = "black") +  
  # Escala de colores accesible  
  scale_fill_manual(values = gender_colors) +  
  # Escala Y con espacio para las etiquetas  
  scale_y_continuous(expand = expansion(mult = c(0, 0.12)),  
    labels = scales::comma) +  
  # Tema personalizado estilo Nature  
  theme_minimal(base_size = 11, base_family = "sans") +  
  theme(  
    # Panel y fondo  
    panel.background = element_rect(fill = "white", color = NA),  
    panel.grid.major.y = element_line(color = "#E0E0E0", linewidth = 0.3),  
    panel.grid.major.x = element_blank(),  
    panel.grid.minor = element_blank(),  
  
    # Bordas de ejes  
    axis.line.x = element_line(color = "black", linewidth = 0.5),  
    axis.line.y = element_line(color = "black", linewidth = 0.5),
```

```

# Títulos y texto
plot.title = element_text(size = 14, face = "bold", hjust = 0,
                           margin = margin(b = 5)),
plot.subtitle = element_text(size = 10, hjust = 0,
                              color = "grey30",
                              margin = margin(b = 10)),

# Ejes
axis.title = element_text(size = 10, face = "bold"),
axis.title.x = element_blank(),
axis.text = element_text(size = 9, color = "black"),
axis.text.x = element_text(size = 10, face = "bold"),
axis.ticks = element_line(color = "black", linewidth = 0.5),

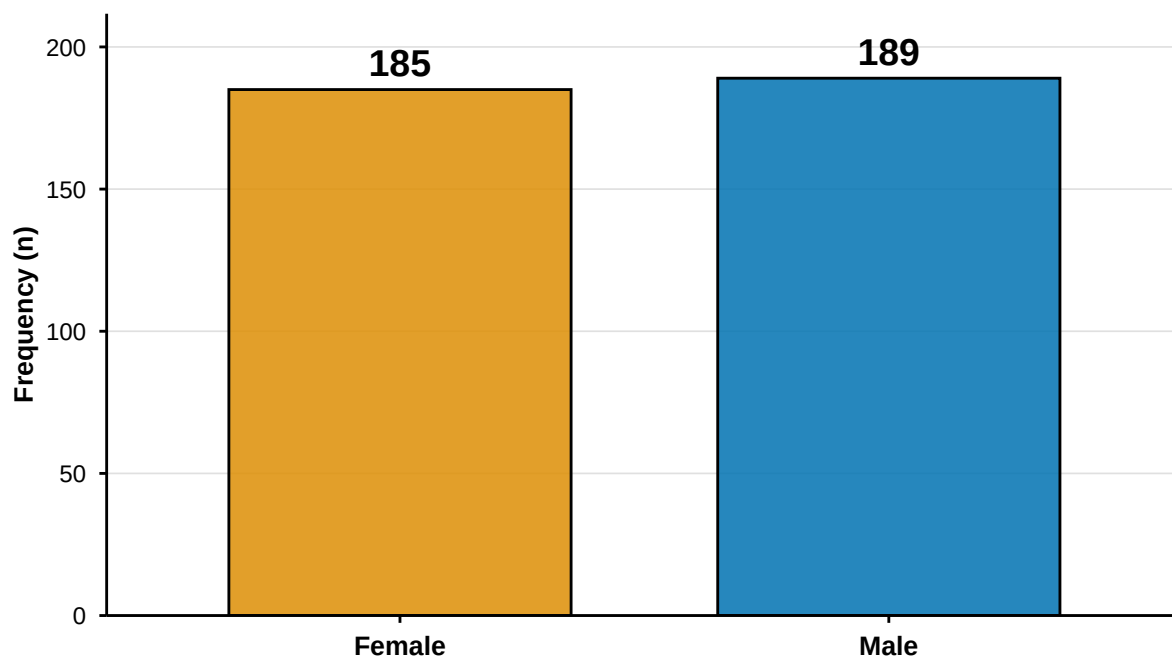
# Sin leyenda
legend.position = "none",

# Márgenes
plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Gender Distribution in Study Population",
  subtitle = "Sample composition by biological sex",
  y = "Frequency (n)",
  x = "",
  caption = "Colorblind-friendly palette"
)

```

Gender Distribution in Study Population

Sample composition by biological sex



Colorblind-friendly palette

```
# BMI Category
cat("\nDistribución por Categoría de IMC:\n")
```

```
##
## Distribución por Categoría de IMC:
```

```
tabla_bmi <- table(df_1$BMI.Category)
print(tabla_bmi)
```

```
##
##      Normal Normal Weight      Obese      Overweight
##      195         21         10         148
```

```
# Paleta accesible para daltonismo - escala secuencial tipo viridis
# Evitamos YlOrRd (rojo-verde problemático) y usamos una escala Blue-Orange
bmi_colors <- c(
  "Underweight" = "#4575B4", # Azul oscuro
  "Normal"      = "#91BFDB", # Azul claro
  "Overweight"  = "#FEE090", # Amarillo
  "Obese"       = "#D73027"  # Naranja-rojizo (OK para daltónicos en esta posición)
)
```

```
# Gráfica
df_1 %>%
```



```

count(BMI.Category) %>%
mutate(BMI.Category = factor(BMI.Category,
                             levels = c("Underweight", "Normal", "Overweight", "Obese"))) %>%

ggplot(aes(x = BMI.Category, y = n, fill = BMI.Category)) +
# Barras con borde negro
geom_col(alpha = 0.85,
         color = "black",
         linewidth = 0.5) +
# Etiquetas de valores con mejor formato
geom_text(aes(label = scales::comma(n)),
         vjust = -0.5,
         size = 4,
         fontface = "bold",
         color = "black") +
# Escala de colores accesible
scale_fill_manual(values = bmi_colors) +
# Escala Y con espacio para las etiquetas
scale_y_continuous(expand = expansion(mult = c(0, 0.1)),
                  labels = scales::comma) +
# Tema personalizado estilo Nature
theme_minimal(base_size = 11, base_family = "sans") +
theme(
  # Panel y fondo
  panel.background = element_rect(fill = "white", color = NA),
  panel.grid.major.y = element_line(color = "#E0E0E0", linewidth = 0.3),
  panel.grid.major.x = element_blank(),
  panel.grid.minor = element_blank(),

  # Bordas
  axis.line.x = element_line(color = "black", linewidth = 0.5),
  axis.line.y = element_line(color = "black", linewidth = 0.5),

  # Títulos y texto
  plot.title = element_text(size = 14, face = "bold", hjust = 0,
                           margin = margin(b = 5)),
  plot.subtitle = element_text(size = 10, hjust = 0,
                              color = "grey30",
                              margin = margin(b = 10)),

  # Ejes
  axis.title = element_text(size = 10, face = "bold"),
  axis.text = element_text(size = 9, color = "black"),
  axis.text.x = element_text(angle = 0, hjust = 0.5),
  axis.ticks = element_line(color = "black", linewidth = 0.5),

  # Sin leyenda
  legend.position = "none",

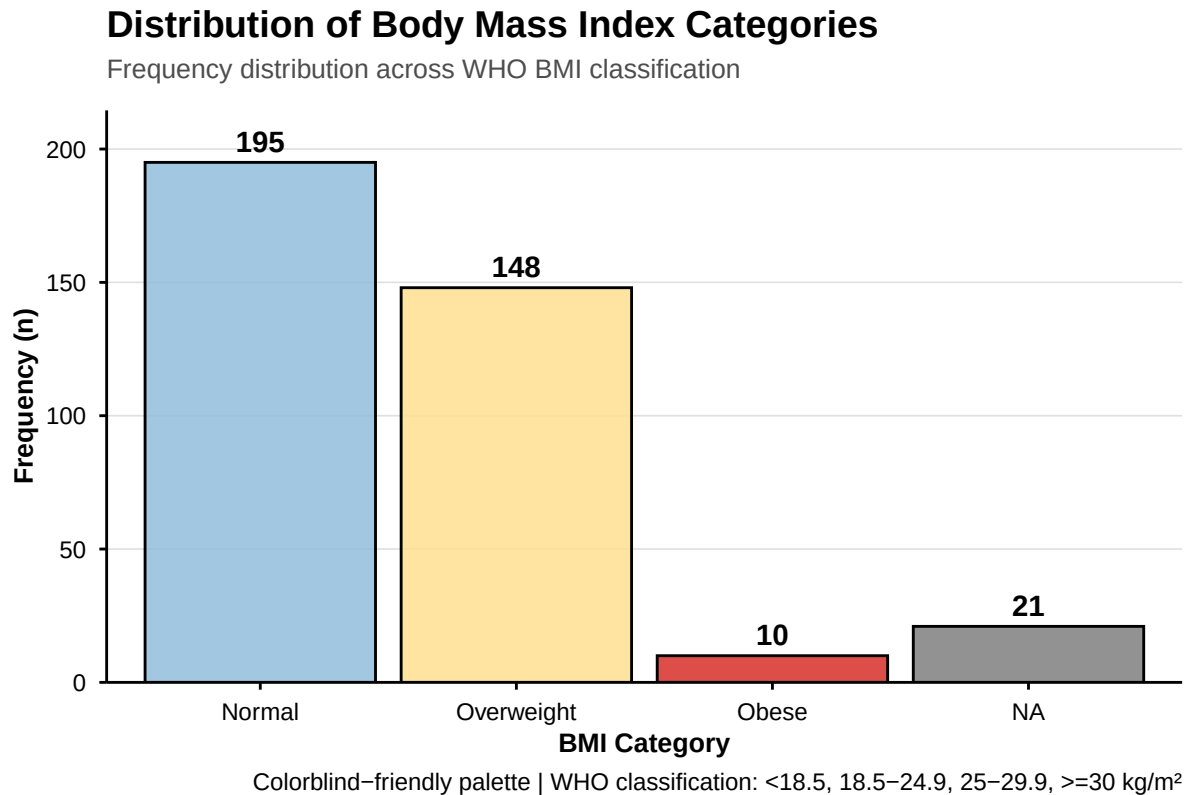
  # Márgenes
  plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Distribution of Body Mass Index Categories",

```

```

subtitle = "Frequency distribution across WHO BMI classification",
y = "Frequency (n)",
x = "BMI Category",
caption = "Colorblind-friendly palette | WHO classification: <18.5, 18.5-24.9, 25-29.9, 30 kg/m²"
)

```



```

cat("\nInterpretación: La distribución del IMC muestra la prevalencia de sobrepeso/obesidad\n")

```

```

##
## Interpretación: La distribución del IMC muestra la prevalencia de sobrepeso/obesidad

```

```

cat("en la población estudiada, factor relevante para trastornos del sueño.\n")

```

```

## en la población estudiada, factor relevante para trastornos del sueño.

```

```

# Sleep Disorder
cat("\nDistribución de Trastornos del Sueño:\n")

```

```

##
## Distribución de Trastornos del Sueño:

```

```
tabla_disorder <- table(df_1$Sleep.Disorder)
print(tabla_disorder)
```

```
##
##      Insomnia      None Sleep Apnea
##          77          219          78
```

```
# Paleta accesible para daltonismo - evitando rojo-verde
sleep_colors <- c(
  "None"      = "#0173B2", # Azul (saludable)
  "Insomnia"   = "#ECE133", # Amarillo brillante (moderado)
  "Sleep Apnea" = "#DE8F05"  # Naranja (severo)
)

# Gráfica
df_1 %>%
  count(Sleep.Disorder) %>%
  ggplot(aes(x = reorder(Sleep.Disorder, -n), y = n, fill = Sleep.Disorder)) +
  # Barras con borde negro definido
  geom_col(alpha = 0.85,
    color = "black",
    linewidth = 0.5) +
  # Etiquetas de valores mejoradas
  geom_text(aes(label = scales::comma(n)),
    vjust = -0.5,
    size = 4.5,
    fontface = "bold",
    color = "black") +
  # Escala de colores accesible
  scale_fill_manual(values = sleep_colors) +
  # Escala Y con espacio para las etiquetas
  scale_y_continuous(expand = expansion(mult = c(0, 0.12)),
    labels = scales::comma) +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    # Panel y fondo
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major.y = element_line(color = "#E0E0E0", linewidth = 0.3),
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),

    # Bordas de ejes
    axis.line.x = element_line(color = "black", linewidth = 0.5),
    axis.line.y = element_line(color = "black", linewidth = 0.5),

    # Títulos y texto
    plot.title = element_text(size = 14, face = "bold", hjust = 0,
      margin = margin(b = 5)),
    plot.subtitle = element_text(size = 10, hjust = 0,
      color = "grey30",
      margin = margin(b = 10)),
```

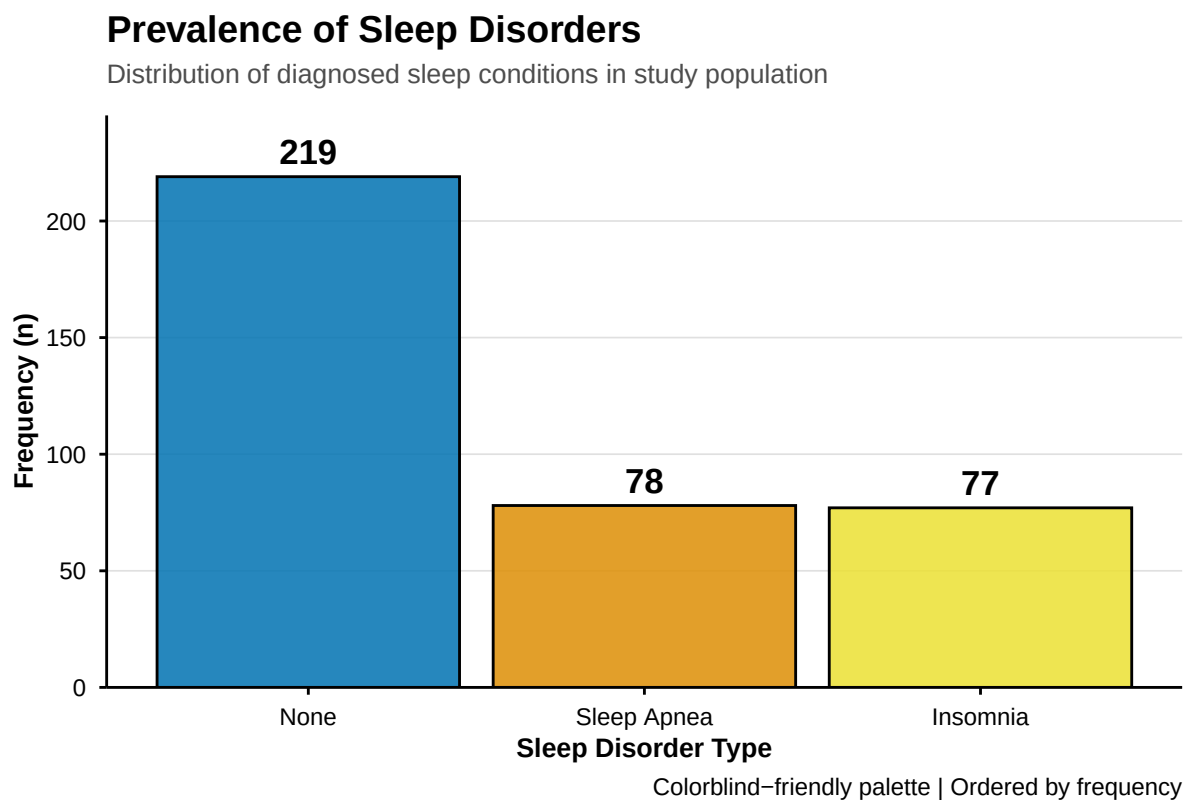
```

# Ejes
axis.title = element_text(size = 10, face = "bold"),
axis.text = element_text(size = 9, color = "black"),
axis.text.x = element_text(angle = 0, hjust = 0.5, size = 9),
axis.ticks = element_line(color = "black", linewidth = 0.5),

# Sin leyenda
legend.position = "none",

# Márgenes
plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Prevalence of Sleep Disorders",
  subtitle = "Distribution of diagnosed sleep conditions in study population",
  y = "Frequency (n)",
  x = "Sleep Disorder Type",
  caption = "Colorblind-friendly palette | Ordered by frequency"
)

```



```
cat("\nInterpretación: La prevalencia de trastornos del sueño indica problemas de salud\n")
```

```
##
## Interpretación: La prevalencia de trastornos del sueño indica problemas de salud
```

```
cat("que requieren atención. Insomnio y apnea son condiciones clínicamente significativas.\n")
```

```
## que requieren atención. Insomnio y apnea son condiciones clínicamente significativas.
```

```
# Occupation (top 10)
```

```
cat("\nTop 10 Ocupaciones más frecuentes:\n")
```

```
##
```

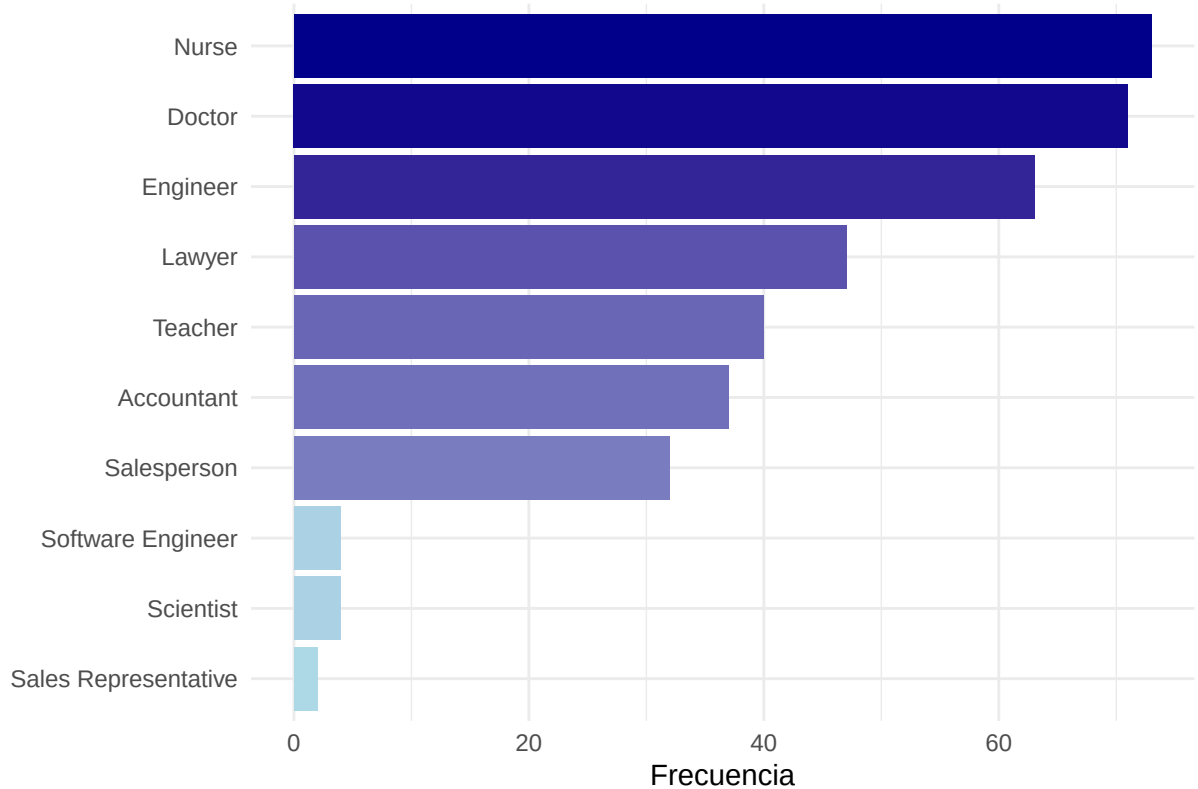
```
## Top 10 Ocupaciones más frecuentes:
```

```
top_ocupaciones <- df_1 %>%  
  count(Occupation, sort = TRUE) %>%  
  head(10)  
print(top_ocupaciones)
```

```
##           Occupation  n  
## 1           Nurse 73  
## 2           Doctor 71  
## 3          Engineer 63  
## 4           Lawyer 47  
## 5           Teacher 40  
## 6       Accountant 37  
## 7       Salesperson 32  
## 8           Scientist 4  
## 9   Software Engineer 4  
## 10 Sales Representative 2
```

```
top_ocupaciones %>%  
  ggplot(aes(x = reorder(Occupation, n), y = n, fill = n)) +  
  geom_col() +  
  coord_flip() +  
  scale_fill_gradient(low = "lightblue", high = "darkblue") +  
  theme_minimal() +  
  labs(title = "Top 10 Ocupaciones en el Dataset",  
        y = "Frecuencia", x = "") +  
  theme(legend.position = "none")
```

Top 10 Ocupaciones en el Dataset



Correlación entre variables

```
matriz_cor <- cor(variables_numericas, use = "complete.obs")
print(round(matriz_cor, 3))
```

```
##           Age Sleep.Duration Quality.of.Sleep
## Age       1.000      0.345      0.474
## Sleep.Duration 0.345      1.000      0.883
## Quality.of.Sleep 0.474      0.883      1.000
## Physical.Activity.Level 0.179      0.212      0.193
## Stress.Level -0.422     -0.811     -0.899
## Heart.Rate -0.226     -0.516     -0.660
## Daily.Steps  0.058     -0.040      0.017
##           Physical.Activity.Level Stress.Level Heart.Rate
## Age              0.179      -0.422     -0.226
## Sleep.Duration    0.212      -0.811     -0.516
## Quality.of.Sleep  0.193      -0.899     -0.660
## Physical.Activity.Level 1.000     -0.034      0.137
## Stress.Level      -0.034      1.000      0.670
## Heart.Rate        0.137      0.670      1.000
## Daily.Steps       0.773      0.187     -0.030
##           Daily.Steps
## Age              0.058
## Sleep.Duration   -0.040
## Quality.of.Sleep  0.017
## Physical.Activity.Level 0.773
```

```
## Stress.Level          0.187
## Heart.Rate            -0.030
## Daily.Steps           1.000
```

```
cat("\nInterpretación de correlaciones destacadas:\n")
```

```
##
## Interpretación de correlaciones destacadas:
```

```
cat("- Valores cercanos a 1 o -1 indican relaciones lineales fuertes.\n")
```

```
## - Valores cercanos a 1 o -1 indican relaciones lineales fuertes.
```

```
cat("- Correlaciones positivas: las variables aumentan conjuntamente.\n")
```

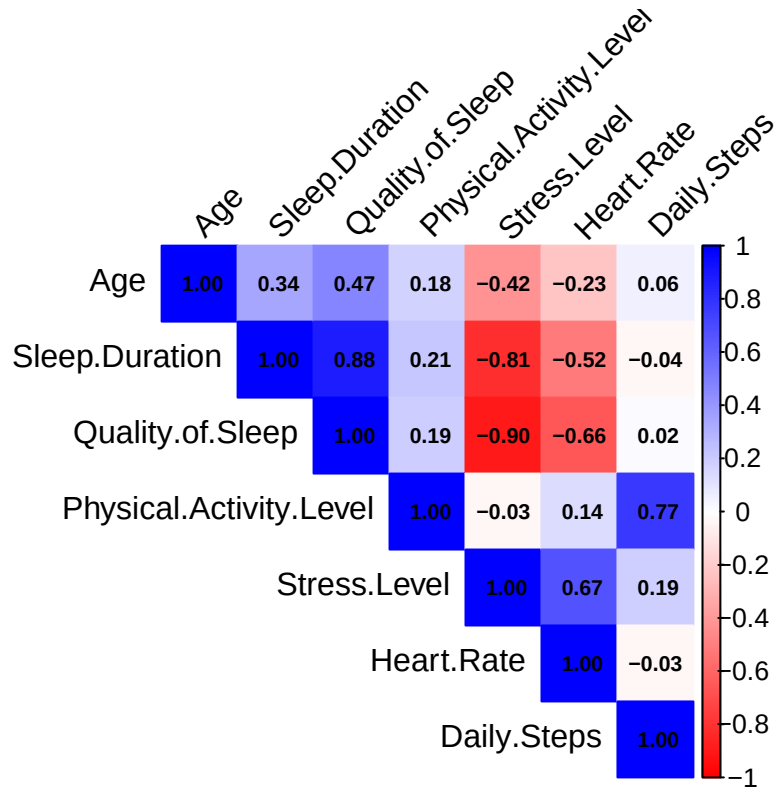
```
## - Correlaciones positivas: las variables aumentan conjuntamente.
```

```
cat("- Correlaciones negativas: una variable aumenta mientras la otra disminuye.\n")
```

```
## - Correlaciones negativas: una variable aumenta mientras la otra disminuye.
```

```
# Visualización con corrplot
corrplot(matriz_cor, method = "color", type = "upper",
         tl.col = "black", tl.srt = 45,
         addCoef.col = "black", number.cex = 0.7,
         col = colorRampPalette(c("red", "white", "blue"))(200),
         title = "Matriz de Correlación",
         mar = c(0,0,2,0))
```

Matriz de Correlación



```
cat("\nCorrelaciones más fuertes detectadas:\n")
```

```
##
```

```
## Correlaciones más fuertes detectadas:
```

```
correlaciones_fuertes <- which(abs(matriz_cor) > 0.5 & matriz_cor != 1, arr.ind = TRUE)
if (nrow(correlaciones_fuertes) > 0) {
  for (i in 1:nrow(correlaciones_fuertes)) {
    row_idx <- correlaciones_fuertes[i, 1]
    col_idx <- correlaciones_fuertes[i, 2]
    if (row_idx < col_idx) { # Evitar duplicados
      cat(paste(rownames(matriz_cor)[row_idx], "~",
                colnames(matriz_cor)[col_idx], ":",
                round(matriz_cor[row_idx, col_idx], 3), "\n"))
    }
  }
} else {
  cat("No se detectaron correlaciones superiores a 0.5.\n")
}
```

```
## Sleep.Duration ~ Quality.of.Sleep : 0.883
## Sleep.Duration ~ Stress.Level : -0.811
## Quality.of.Sleep ~ Stress.Level : -0.899
## Sleep.Duration ~ Heart.Rate : -0.516
## Quality.of.Sleep ~ Heart.Rate : -0.66
```



```
## Stress.Level ~ Heart.Rate : 0.67
## Physical.Activity.Level ~ Daily.Steps : 0.773
```

Observando el dataset y la matriz de correlación, hemos percibido que dos variables (Sleep Quality y Sleep Duration) podrían estar relacionadas, o que, podrían estar inferidas una a través de la otra. Esto podría suponer problemas posteriores a la hora de entrenar modelos o sacar insights, ya que las variables tienen relación entre ellas. Por lo tanto, hemos decidido corroborar si estas variables, Sleep Quality y Sleep Duration están totalmente correlacionadas.

```
# Correlación de Pearson (asume cierta linealidad)
cor_pearson <- cor(df_1$Sleep.Duration, df_1$Quality.of.Sleep, use = "complete.obs", method = "pearson")

cat("Correlación Pearson:", round(cor_pearson, 3), "\n")
```

```
## Correlación Pearson: 0.883
```

```
# Colores accesibles para daltonismo
color_points <- "#0173B2"      # Azul para puntos
color_regression <- "#DE8F05"  # Naranja para línea de regresión
color_ci <- "#FDB462"          # Naranja claro para intervalo de confianza

# Calcular correlación para incluir en el gráfico
correlation <- cor(df_1$Sleep.Duration, df_1$Quality.of.Sleep, use = "complete.obs")
correlation_text <- paste0("r = ", round(correlation, 3))

# Ajustar modelo para estadísticas
model <- lm(Quality.of.Sleep ~ Sleep.Duration, data = df_1)
r_squared <- summary(model)$r.squared
p_value <- summary(model)$coefficients[2, 4]

# Texto de estadísticas
stats_text <- paste0(
  "r = ", round(correlation, 3),
  " (R² = ", round(r_squared, 3), ")",
  ifelse(p_value < 0.001, "\np < 0.001", paste0("\np = ", round(p_value, 3)))
)

# Gráfica
ggplot(df_1, aes(x = Sleep.Duration, y = Quality.of.Sleep)) +
  # Intervalo de confianza primero (detrás)
  geom_smooth(method = "lm",
    se = TRUE,
    color = color_regression,
    fill = color_ci,
    alpha = 0.3,
    linewidth = 1.2) +
  # Puntos encima
  geom_point(color = color_points,
    alpha = 0.5,
    size = 2.5,
    shape = 16) +
  # Añadir estadísticas en el gráfico
  annotate("text",
```

```

    x = min(df_1$Sleep.Duration, na.rm = TRUE) + 0.5,
    y = max(df_1$Quality.of.Sleep, na.rm = TRUE) - 0.5,
    label = stats_text,
    hjust = 0,
    vjust = 1,
    size = 3.5,
    fontface = "bold",
    color = "black") +
# Tema personalizado estilo Nature
theme_minimal(base_size = 11, base_family = "sans") +
theme(
  # Panel y fondo
  panel.background = element_rect(fill = "white", color = NA),
  panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

  # Títulos y texto
  plot.title = element_text(size = 14, face = "bold", hjust = 0,
                             margin = margin(b = 5)),
  plot.subtitle = element_text(size = 10, hjust = 0,
                                color = "grey30",
                                margin = margin(b = 10)),

  # Ejes
  axis.title = element_text(size = 10, face = "bold"),
  axis.text = element_text(size = 9, color = "black"),
  axis.line = element_line(color = "black", linewidth = 0.5),
  axis.ticks = element_line(color = "black", linewidth = 0.5),

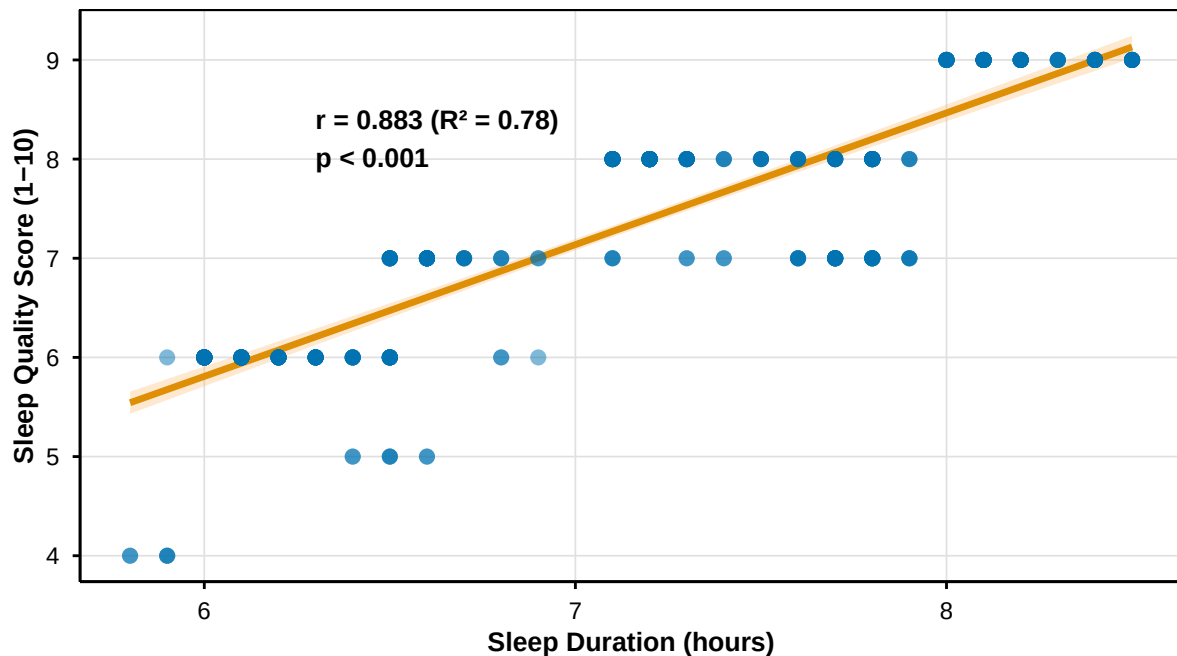
  # Márgenes
  plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Relationship Between Sleep Duration and Sleep Quality",
  subtitle = "Linear regression with 95% confidence interval",
  x = "Sleep Duration (hours)",
  y = "Sleep Quality Score (1-10)",
  caption = "Colorblind-friendly palette | Pearson correlation coefficient shown"
)

```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Relationship Between Sleep Duration and Sleep Quality

Linear regression with 95% confidence interval



Colorblind-friendly palette | Pearson correlation coefficient shown

Aplicando correlación de Pearson, vemos que las variables tienen una correlación de 0.8.

Cuando una variable está inferida de otra, se introduce colinealidad perfecta o casi perfecta, y por ende, en los modelos de regresión, esto distorsiona las estimaciones de los coeficientes, porque el modelo no puede distinguir qué variable está aportando realmente la información.

En análisis exploratorio, puede duplicar información sin añadir valor.

Una vez hacemos el test de correlación, aunque la correlación entre la duración del sueño y la calidad del sueño es alta ($r = 0.8$), esta correlación no es perfecta, lo que indica que ambas variables están relacionadas, pero no miden exactamente el mismo fenómeno.

Una correlación de 0.8 no indica colinealidad perfecta, por lo tanto si eliminamos una de ellas provocaría pérdida de información relevante. Además, representan aspectos diferentes del sueño, uno más objetivo y otro más subjetivo. En resumen, nos quedamos con ambas variables para realizar el estudio.

1. ¿Existen diferencias en la duración media del sueño entre hombres y mujeres?

Hemos observado en diferentes estudios estadísticos que la duración del sueño puede variar entre hombres y mujeres. Por lo tanto, hemos decidido observar si este fenómeno se produce en nuestro estudio.

Como hipótesis inicial a demostrar, queremos comprobar si las mujeres suelen reportar una mayor duración del sueño frente a una duración por parte de los hombres, ya sea por motivos laborales o estilos de vida.

```
# Separar datos por género
sleep_male <- df_1$Sleep.Duration[df_1$Gender == "Male"]
```

```
sleep_female <- df_1$Sleep.Duration[df_1$Gender == "Female"]

# Estadísticas descriptivas:
# MEDIA
cat("\nEstadísticas descriptivas:")
```

Comprobación de Normalidad

```
##
## Estadísticas descriptivas:

cat("Hombres - Media:", mean(sleep_male, na.rm = TRUE), "| SD:", sd(sleep_male, na.rm = TRUE), "\n")

## Hombres - Media: 7.036508 | SD: 0.6931968

cat("Mujeres - Media:", mean(sleep_female, na.rm = TRUE), "| SD:", sd(sleep_female, na.rm = TRUE), "\n")

## Mujeres - Media: 7.22973 | SD: 0.8794344

# MEDIANA
cat("Hombres - Mediana:", median(sleep_male, na.rm = TRUE), "| SD:", sd(sleep_male, na.rm = TRUE), "\n")

## Hombres - Mediana: 7.2 | SD: 0.6931968

cat("Mujeres - Mediana:", median(sleep_female, na.rm = TRUE), "| SD:", sd(sleep_female, na.rm = TRUE), "\n")

## Mujeres - Mediana: 7.2 | SD: 0.8794344

# Test de normalidad (Shapiro-Wilk)
cat("\nTest de normalidad (Shapiro-Wilk):")

##
## Test de normalidad (Shapiro-Wilk):

shapiro_male <- shapiro.test(sleep_male)
shapiro_female <- shapiro.test(sleep_female)
cat("Hombres p-value:", shapiro_male$p.value, "\n")

## Hombres p-value: 1.51263e-11

cat("Mujeres p-value:", shapiro_female$p.value, "\n")

## Mujeres p-value: 6.360571e-10
```

Dado que los datos, tanto en hombres como en mujeres no siguen una distribución normal, tendríamos que aplicar el test de Wilcoxon. Pero antes, dibujamos las funciones de densidad de los datos.

```

# Paleta accesible para daltonismo (igual que gráfica de género)
gender_colors <- c(
  "Female" = "#DE8F05", # Naranja
  "Male"   = "#0173B2"  # Azul
)

# Gráfica en estilo Nature
p_density <- ggplot(df_1, aes(x = Sleep.Duration, fill = Gender, color = Gender)) +
  # Curvas de densidad con relleno y borde
  geom_density(alpha = 0.4,
    linewidth = 1.2) +
  # Escala de colores accesible
  scale_fill_manual(values = gender_colors,
    name = "Gender") +
  scale_color_manual(values = gender_colors,
    name = "Gender") +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    # Panel y fondo
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

    # Títulos y texto
    plot.title = element_text(size = 14, face = "bold", hjust = 0,
      margin = margin(b = 5)),
    plot.subtitle = element_text(size = 10, hjust = 0,
      color = "grey30",
      margin = margin(b = 10)),

    # Ejes
    axis.title = element_text(size = 10, face = "bold"),
    axis.text = element_text(size = 9, color = "black"),
    axis.line = element_line(color = "black", linewidth = 0.5),
    axis.ticks = element_line(color = "black", linewidth = 0.5),

    # Leyenda
    legend.position = "top",
    legend.justification = "left",
    legend.title = element_text(size = 10, face = "bold"),
    legend.text = element_text(size = 9),
    legend.background = element_rect(fill = "white", color = "black", linewidth = 0.3),
    legend.key.size = unit(0.8, "cm"),
    legend.margin = margin(5, 5, 5, 5),
    legend.box.spacing = unit(0.5, "cm"),

    # Márgenes
    plot.margin = margin(15, 15, 15, 15)
  ) +
  labs(

```

```

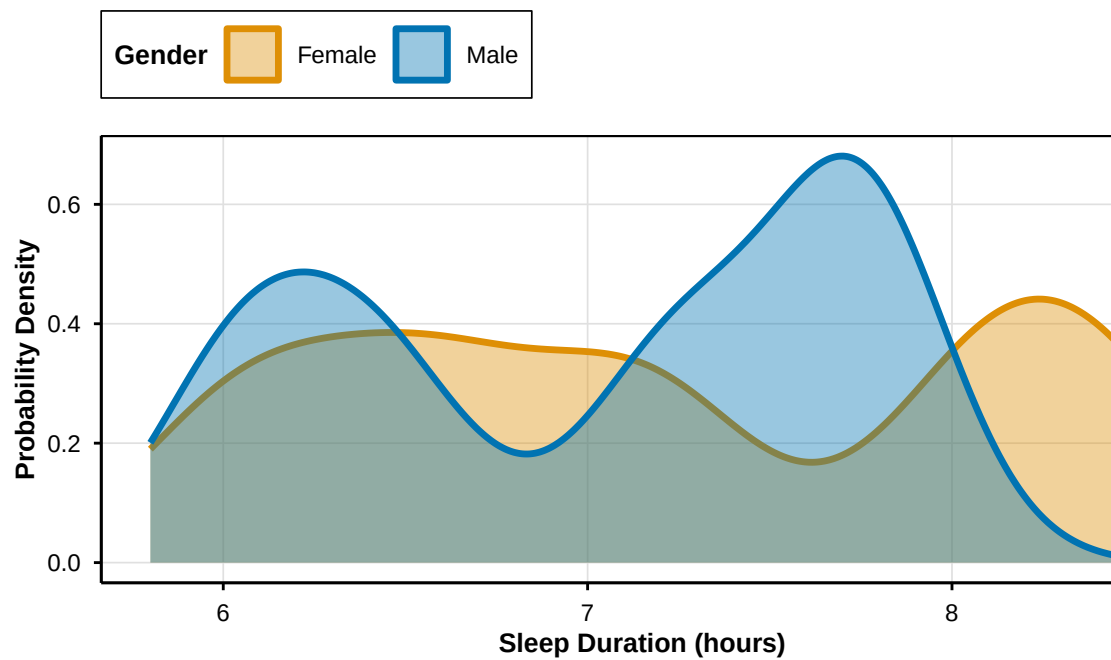
title = "Sleep Duration Distribution by Gender",
subtitle = "Kernel density estimation with overlapping comparison",
x = "Sleep Duration (hours)",
y = "Probability Density",
caption = "Colorblind-friendly palette | Shaded areas show distribution overlap"
)

# Mostrar la gráfica
print(p_density)

```

Sleep Duration Distribution by Gender

Kernel density estimation with overlapping comparison



Colorblind-friendly palette | Shaded areas show distribution overlap

Gráficos de Densidad

Podemos observar que los datos, tanto los de las mujeres como los de los hombres, siguen un comportamiento bimodal. Debido a esto, hemos decidido comprobar a que se debe esa diferencia en la duración del sueño. Por lo tanto, realizamos un análisis de Clustering mediante K-Means.

```

# Para HOMBRES
sleep_male <- df_1$Sleep.Duration[df_1$Gender == "Male"]

# K-means (2 clusters)
set.seed(123) # para reproducibilidad
kmeans_male <- kmeans(sleep_male, centers = 2, nstart = 25)

df_1$Cluster_Male <- NA
df_1$Cluster_Male[df_1$Gender == "Male"] <- kmeans_male$cluster

```

```
# Para MUJERES
sleep_female <- df_1$Sleep.Duration[df_1$Gender == "Female"]
kmeans_female <- kmeans(sleep_female, centers = 2, nstart = 25)

df_1$Cluster_Female <- NA
df_1$Cluster_Female[df_1$Gender == "Female"] <- kmeans_female$cluster
```

CLUSTERING CON K-MEANS

CARACTERÍSTICAS DE LOS CLUSTERS HOMBRES

```
cat("Cluster 1 - Media:", round(mean(sleep_male[kmeans_male$cluster == 1]), 2), "h\n")
```

```
## Cluster 1 - Media: 6.26 h
```

```
cat("Cluster 2 - Media:", round(mean(sleep_male[kmeans_male$cluster == 2]), 2), "h\n")
```

```
## Cluster 2 - Media: 7.57 h
```

MUJERES

```
cat("Cluster 1 - Media:", round(mean(sleep_female[kmeans_female$cluster == 1]), 2), "h\n")
```

```
## Cluster 1 - Media: 8.23 h
```

```
cat("Cluster 2 - Media:", round(mean(sleep_female[kmeans_female$cluster == 2]), 2), "h\n")
```

```
## Cluster 2 - Media: 6.61 h
```

```
# Preparamos los datos para graficar
df_1_plot <- df_1 %>%
  filter(!is.na(Cluster_Male) | !is.na(Cluster_Female)) %>%
  mutate(
    Cluster = ifelse(Gender == "Male", Cluster_Male, Cluster_Female),
    Cluster = as.factor(Cluster)
  )

# Paleta accesible para daltonismo - 3 colores distinguibles
cluster_colors <- c(
  "1" = "#0173B2", # Azul
  "2" = "#ECE133", # Amarillo
  "3" = "#DE8F05"  # Naranja
)

# Si tienes más de 3 clusters, usar esta paleta extendida:
# cluster_colors <- c("#0173B2", "#ECE133", "#DE8F05", "#CC78BC", "#029E73")
```

```

# Gráfica en estilo Nature
ggplot(df_1_plot, aes(x = Sleep.Duration, fill = Cluster)) +
  # Curvas de densidad con ajuste suavizado
  geom_density(alpha = 0.5,
               adjust = 1.2,
               color = "black",
               linewidth = 0.5) +
  # Facetas por género
  facet_wrap(~ Gender, scales = "free", ncol = 2) +
  # Escala de colores accesible
  scale_fill_manual(values = cluster_colors,
                   name = "Cluster") +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    # Panel y fondo
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

    # Títulos y texto
    plot.title = element_text(size = 14, face = "bold", hjust = 0,
                              margin = margin(b = 5)),
    plot.subtitle = element_text(size = 10, hjust = 0,
                                 color = "grey30",
                                 margin = margin(b = 10)),

    # Etiquetas de facetas
    strip.background = element_rect(fill = "#F0F0F0",
                                    color = "black",
                                    linewidth = 0.5),
    strip.text = element_text(size = 10, face = "bold",
                              margin = margin(3, 3, 3, 3)),

    # Ejes
    axis.title = element_text(size = 10, face = "bold"),
    axis.text = element_text(size = 9, color = "black"),
    axis.line = element_line(color = "black", linewidth = 0.5),
    axis.ticks = element_line(color = "black", linewidth = 0.5),

    # Leyenda
    legend.position = "top",
    legend.justification = "left",
    legend.title = element_text(size = 10, face = "bold"),
    legend.text = element_text(size = 9),
    legend.background = element_rect(fill = "white", color = "black", linewidth = 0.3),
    legend.key.size = unit(0.7, "cm"),
    legend.margin = margin(5, 5, 5, 5),
    legend.box.spacing = unit(0.3, "cm"),

    # Márgenes
    plot.margin = margin(15, 15, 15, 15)
  )

```



```

) +
labs(
  title = "Sleep Duration Distribution by K-means Cluster and Gender",
  subtitle = "Kernel density estimation showing cluster separation within gender groups",
  x = "Sleep Duration (hours)",
  y = "Probability Density",
  fill = "Cluster",
  caption = "Colorblind-friendly palette | Bandwidth multiplier = 1.2"
)

```

Sleep Duration Distribution by K-means Cluster and Gender

Kernel density estimation showing cluster separation within gender groups

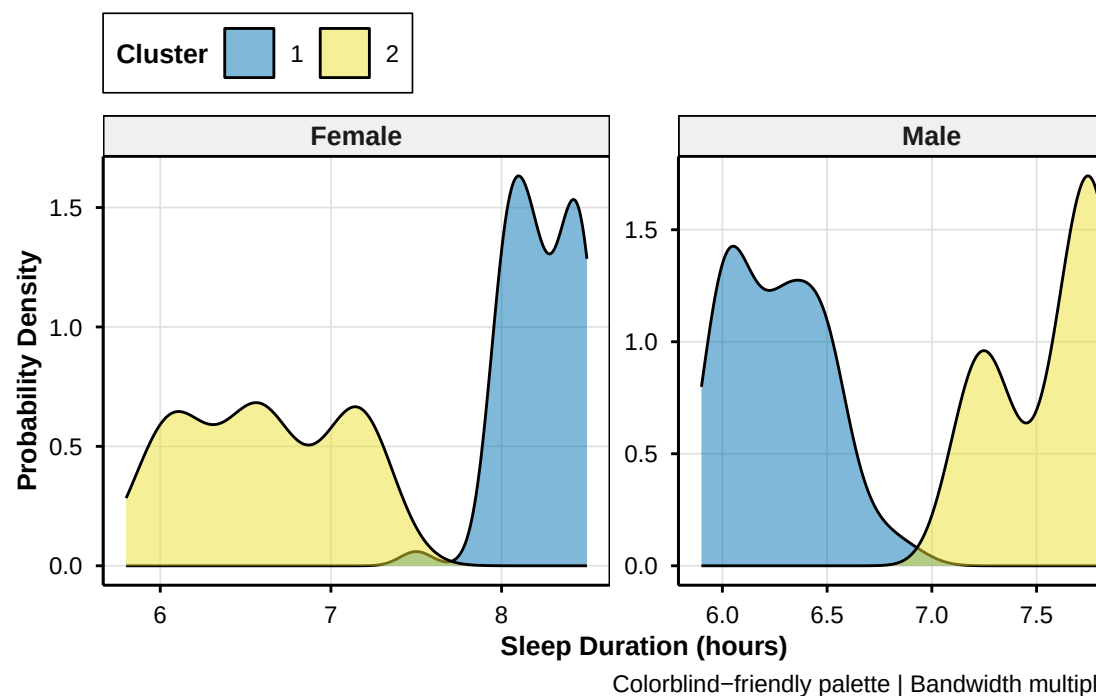


Gráfico de los Clusters

Una vez separado por clústers, vamos a observar qué variables són las que más contribuyen a la formación de estos clústers.

MODELO LASSO PARA EXPLICAR CLÚSTERS HOMBRES

```

df_1_male <- df_1 %>%
  filter(Gender == "Male") %>%
  mutate(Cluster = as.factor(Cluster_Male)) %>%
  select(Cluster, Age, Quality.of.Sleep, Physical.Activity.Level,
         Stress.Level, Heart.Rate, Daily.Steps) %>%
  na.omit()

X_male <- model.matrix(Cluster ~ . - 1, data = df_1_male)
y_male <- as.numeric(df_1_male$Cluster) - 1 # Convertir a 0/1

```

```
lasso_male <- cv.glmnet(X_male, y_male, alpha = 1, family = "binomial")
coef_male <- coef(lasso_male, s = "lambda.min")
```

VARIABLES QUE EXPLICAN BIMODALIDAD EN HOMBRES

```
print(coef_male[coef_male[,1] != 0, , drop = FALSE])
```

```
## 6 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.min
## (Intercept)    -49.57403526
## Age           0.10807321
## Quality.of.Sleep 4.73723819
## Physical.Activity.Level 0.43058902
## Stress.Level   -1.24955589
## Heart.Rate     -0.02342066
```

MUJERES

```
df_1_female <- df_1 %>%
  filter(Gender == "Female") %>%
  mutate(Cluster = as.factor(Cluster_Female)) %>%
  select(Cluster, Age, Quality.of.Sleep, Physical.Activity.Level,
         Stress.Level, Heart.Rate, Daily.Steps) %>%
  na.omit()

X_female <- model.matrix(Cluster ~ . - 1, data = df_1_female)
y_female <- as.numeric(df_1_female$Cluster) - 1

lasso_female <- cv.glmnet(X_female, y_female, alpha = 1, family = "binomial")
coef_female <- coef(lasso_female, s = "lambda.min")
```

VARIABLES QUE EXPLICAN BIMODALIDAD EN MUJERES

```
print(coef_female[coef_female[,1] != 0, , drop = FALSE])
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.min
## (Intercept)    49.23578778
## Age          -0.09907425
## Quality.of.Sleep -5.38931041
## Stress.Level    0.01580211
```

Podemos observar que la variable que más influye en la duración del sueño para hombres es la **calidad del sueño** y el **estrés**. En cambio, en las mujeres, variable de la **calidad del sueño** es la que más influye, mayoritariamente.

Para ver si realmente la calidad del sueño es la variable que más afecta en la creación de clústers tanto en hombre como en mujeres, vamos a calcular la mas medias de esta variable para cada clúster a ver si se corresponde con la media de los clústers.

HOMBRES

```
media_calidad_hombres <- df_1 %>%
  filter(Gender == "Male", !is.na(Cluster_Male)) %>%
  group_by(Cluster_Male) %>%
  summarise(
    Media_Calidad_Sueno = mean(Quality.of.Sleep, na.rm = TRUE),
    n = n()
  )
```

MUJERES

```
media_calidad_mujeres <- df_1 %>%
  filter(Gender == "Female", !is.na(Cluster_Female)) %>%
  group_by(Cluster_Female) %>%
  summarise(
    Media_Calidad_Sueno = mean(Quality.of.Sleep, na.rm = TRUE),
    n = n()
  )
```

MEDIA DE CALIDAD DEL SUEÑO POR CLÚSTER HOMBRES

```
print(media_calidad_hombres)
```

```
## # A tibble: 2 x 3
##   Cluster_Male Media_Calidad_Sueno      n
##         <int>             <dbl> <int>
## 1           1             5.92     77
## 2           2             7.69    112
```

MUJERES

```
print(media_calidad_mujeres)
```

```
## # A tibble: 2 x 3
##   Cluster_Female Media_Calidad_Sueno      n
##         <int>             <dbl> <int>
## 1           1             8.97     71
## 2           2             6.85    114
```

Efectivamente, podemos observar que las medias de la calidad del sueño se corresponden casi con exactitud con las medias de los clústers. Conclusión: la calidad del sueño es la variable que más interviene en la formación de clústers.

Ahora realizamos una comparación entre géneros de las variables más importantes.

```
vars_male <- rownames(coef_male)[coef_male[,1] != 0 &
  rownames(coef_male) != "(Intercept)"]
vars_female <- rownames(coef_female)[coef_female[,1] != 0 &
  rownames(coef_female) != "(Intercept)"]

cat("Variables importantes en HOMBRES:", paste(vars_male, collapse = ", "), "\n")
```

COMPARACIÓN DE VARIABLES IMPORTANTES

```
## Variables importantes en HOMBRES: Age, Quality.of.Sleep, Physical.Activity.Level, Stress.Level, Heart.Rate
```

```
cat("Variables importantes en MUJERES:", paste(vars_female, collapse = ", "), "\n")
```

```
## Variables importantes en MUJERES: Age, Quality.of.Sleep, Stress.Level
```

```
cat("\nComunes:", paste(intersect(vars_male, vars_female), collapse = ", "), "\n")
```

```
##
```

```
## Comunes: Age, Quality.of.Sleep, Stress.Level
```

```
cat("Solo hombres:", paste(setdiff(vars_male, vars_female), collapse = ", "), "\n")
```

```
## Solo hombres: Physical.Activity.Level, Heart.Rate
```

```
cat("Solo mujeres:", paste(setdiff(vars_female, vars_male), collapse = ", "), "\n")
```

```
## Solo mujeres:
```

Podemos observar que las variables de Nivel de Actividad Física y Frecuencia Cardíaca influyen solo en los hombres.

El siguiente paso es observar si las distribuciones separadas por clústers siguen una distribución normal y, acto seguido, hacer los test de hipótesis correspondientes.

HOMBRES

SHAPIRO-WILK TEST

```
for (cl in sort(unique(na.omit(df_1$Cluster_Male)))) {
  valores <- df_1$Quality.of.Sleep[df_1$Gender == "Male" & df_1$Cluster_Male == cl]
  if (length(valores) >= 3) { # Shapiro requiere >= 3 valores
    cat("\nCluster", cl, ":\n")
    print(shapiro.test(valores))
  } else {
    cat("\nCluster", cl, ": No hay suficientes datos para hacer el test.\n")
  }
}
```

```
##
```

```
## Cluster 1 :
```

```
##
```

```
## Shapiro-Wilk normality test
```

```
##
```

```
## data: valores
```

```
## W = 0.43782, p-value = 1.204e-15
```

```
##
```

```
##
```

```
## Cluster 2 :
```

```
##
```

```
## Shapiro-Wilk normality test
##
## data:  valores
## W = 0.66033, p-value = 9.794e-15
```

MUJERES

SHAPIRO-WILK TEST

```
for (cl in sort(unique(na.omit(df_1$Cluster_Female)))) {
  valores <- df_1$Quality.of.Sleep[df_1$Gender == "Female" & df_1$Cluster_Female == cl]
  if (length(valores) >= 3) {
    cat("\nCluster", cl, ":\n")
    print(shapiro.test(valores))
  } else {
    cat("\nCluster", cl, ": No hay suficientes datos para hacer el test.\n")
  }
}
```

```
##
## Cluster 1 :
##
## Shapiro-Wilk normality test
##
## data:  valores
## W = 0.15613, p-value < 2.2e-16
##
##
## Cluster 2 :
##
## Shapiro-Wilk normality test
##
## data:  valores
## W = 0.85762, p-value = 4.426e-09
```

Podemos observar que los shapiro test de cada uno de los clústers nos sale significativo, por lo tanto, rechazamos normalidad. Realizamos test de Wilcoxon.

```
# Detectamos cuál clúster tiene menor media de horas de sueño para HOMBRES
mean_male_cluster1 <- mean(df_1$Sleep.Duration[df_1$Gender == "Male" & df_1$Cluster_Male == 1], na.rm =
mean_male_cluster2 <- mean(df_1$Sleep.Duration[df_1$Gender == "Male" & df_1$Cluster_Male == 2], na.rm =
cluster_bajo_male <- ifelse(mean_male_cluster1 < mean_male_cluster2, 1, 2)

# Detectamos cuál clúster tiene menor media para MUJERES
mean_female_cluster1 <- mean(df_1$Sleep.Duration[df_1$Gender == "Female" & df_1$Cluster_Female == 1], na.rm =
mean_female_cluster2 <- mean(df_1$Sleep.Duration[df_1$Gender == "Female" & df_1$Cluster_Female == 2], na.rm =
cluster_bajo_female <- ifelse(mean_female_cluster1 < mean_female_cluster2, 1, 2)

# Wilcoxon para clúster de BAJO sueño
wilcox_bajo <- wilcox.test(
  df_1$Sleep.Duration[df_1$Gender == "Male" & df_1$Cluster_Male == cluster_bajo_male],
```

```
df_1$Sleep.Duration[df_1$Gender == "Female" & df_1$Cluster_Female == cluster_bajo_female]
)
```

WILCOXON: BAJA DURACIÓN DEL SUEÑO WILCOXON: BAJA DURACIÓN DEL SUEÑO HOMBRES vs MUJERES

```
print(wilcox_bajo)
```

```
##
## Wilcoxon rank sum test with continuity correction
##
## data: df_1$Sleep.Duration[df_1$Gender == "Male" & df_1$Cluster_Male == cluster_bajo_male] and df_1$
## W = 2406.5, p-value = 1.023e-07
## alternative hypothesis: true location shift is not equal to 0
```

```
# Clúster con mayor media de sueño
cluster_alto_male <- ifelse(mean_male_cluster1 > mean_male_cluster2, 1, 2)
cluster_alto_female <- ifelse(mean_female_cluster1 > mean_female_cluster2, 1, 2)

# Wilcoxon para clúster de ALTO sueño
wilcox_alto <- wilcox.test(
  df_1$Sleep.Duration[df_1$Gender == "Male" & df_1$Cluster_Male == cluster_alto_male],
  df_1$Sleep.Duration[df_1$Gender == "Female" & df_1$Cluster_Female == cluster_alto_female]
)
```

WILCOXON: ALTA DURACIÓN DEL SUEÑO WILCOXON: ALTA DURACIÓN DEL SUEÑO HOMBRES vs MUJERES

```
print(wilcox_alto)
```

```
##
## Wilcoxon rank sum test with continuity correction
##
## data: df_1$Sleep.Duration[df_1$Gender == "Male" & df_1$Cluster_Male == cluster_alto_male] and df_1$
## W = 116, p-value < 2.2e-16
## alternative hypothesis: true location shift is not equal to 0
```

Ambos test de Wilcoxon són significativos, tanto el test que compara los clústers asociados a una baja duración del sueño, como los asociados a una alta duración del sueño.

Habiendo separado las distribuciones según bimodalidad y genero, podemos afirmar que las medias no són iguales. Las medias de duración del sueño de las mujeres son ligeramente superiores.

2. ¿La calidad del sueño difiere según la categoría de IMC?

```
# Verificar categorías de IMC antes de agrupar
cat("\nCategorías de IMC antes de agrupar:\n")
```

```
##
## Categorías de IMC antes de agrupar:

table(df_1$BMI.Category)

##
##      Normal Normal Weight      Obese      Overweight
##      195         21         10         148

# Agrupar "Normal Weight" y "Normal" bajo la misma etiqueta "Normal"
df_1$BMI.Category <- ifelse(df_1$BMI.Category %in% c("Normal", "Normal Weight"), "Normal", df_1$BMI.Cat

# Verificar categorías después de agrupar
cat("\nCategorías de IMC después de agrupar:\n")

##
## Categorías de IMC después de agrupar:

table(df_1$BMI.Category)

##
##      Normal      Obese Overweight
##      216         10         148

# Estadísticas descriptivas por grupo
cat("\nEstadísticas descriptivas por categoría de IMC:\n")

##
## Estadísticas descriptivas por categoría de IMC:

print(aggregate(Quality.of.Sleep ~ BMI.Category, data = df_1, FUN = function(x) c(mean = mean(x), sd =

##      BMI.Category Quality.of.Sleep.mean Quality.of.Sleep.sd Quality.of.Sleep.n
## 1      Normal          7.6388889          0.9972831          216.0000000
## 2      Obese           6.4000000          1.8973666           10.0000000
## 3 Overweight          6.8986486          1.2548673          148.0000000

#### Normalidad
cat("\n Verificando normalidad con Shapiro-Wilk por grupo \n")

##
## Verificando normalidad con Shapiro-Wilk por grupo

by(df_1$Quality.of.Sleep, df_1$BMI.Category, function(x) shapiro.test(x))

## df_1$BMI.Category: Normal
##
## Shapiro-Wilk normality test
##
```

```

## data:  x
## W = 0.85021, p-value = 1.144e-13
##
## -----
## df_1$BMI.Category: Obese
##
## Shapiro-Wilk normality test
##
## data:  x
## W = 0.8602, p-value = 0.07673
##
## -----
## df_1$BMI.Category: Overweight
##
## Shapiro-Wilk normality test
##
## data:  x
## W = 0.78889, p-value = 2.482e-13

# Paleta accesible para daltonismo (misma que BMI anterior)
bmi_colors <- c(
  "Underweight" = "#4575B4", # Azul oscuro
  "Normal"       = "#91BFDB", # Azul claro
  "Overweight"   = "#FEE090", # Amarillo
  "Obese"        = "#D73027"  # Naranja-rojizo
)

# Preparar datos para QQ plots
qq_data <- df_1 %>%
  filter(!is.na(Quality.of.Sleep), !is.na(BMI.Category)) %>%
  group_by(BMI.Category) %>%
  arrange(Quality.of.Sleep) %>%
  mutate(
    theoretical = qqnorm(Quality.of.Sleep, plot.it = FALSE)$x,
    sample = qqnorm(Quality.of.Sleep, plot.it = FALSE)$y
  ) %>%
  ungroup() %>%
  mutate(BMI.Category = factor(BMI.Category,
                               levels = c("Underweight", "Normal", "Overweight", "Obese")))

# Calcular líneas de referencia para cada categoría
qq_lines <- qq_data %>%
  group_by(BMI.Category) %>%
  summarise(
    slope = (quantile(sample, 0.75) - quantile(sample, 0.25)) /
             (qnorm(0.75) - qnorm(0.25)),
    intercept = quantile(sample, 0.25) - slope * qnorm(0.25)
  )

# Gráfica
ggplot(qq_data, aes(x = theoretical, y = sample)) +
  # Puntos de los datos
  geom_point(aes(color = BMI.Category, fill = BMI.Category),
             size = 2,

```



```

        alpha = 0.6,
        shape = 21,
        stroke = 0.3) +
# Línea de referencia teórica
geom_abline(data = qq_lines,
            aes(slope = slope, intercept = intercept),
            color = "black",
            linewidth = 0.8,
            linetype = "solid") +
# Facetas por categoría BMI
facet_wrap(~ BMI.Category, scales = "free", ncol = 2) +
# Escalas de colores
scale_color_manual(values = bmi_colors) +
scale_fill_manual(values = bmi_colors) +
# Tema personalizado estilo Nature
theme_minimal(base_size = 11, base_family = "sans") +
theme(
  # Panel y fondo
  panel.background = element_rect(fill = "white", color = NA),
  panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

  # Títulos y texto
  plot.title = element_text(size = 14, face = "bold", hjust = 0,
                           margin = margin(b = 5)),
  plot.subtitle = element_text(size = 10, hjust = 0,
                              color = "grey30",
                              margin = margin(b = 10)),

  # Etiquetas de facetas
  strip.background = element_rect(fill = "#F0F0F0",
                                   color = "black",
                                   linewidth = 0.5),
  strip.text = element_text(size = 10, face = "bold",
                            margin = margin(3, 3, 3, 3)),

  # Ejes
  axis.title = element_text(size = 10, face = "bold"),
  axis.text = element_text(size = 8, color = "black"),
  axis.line = element_line(color = "black", linewidth = 0.5),
  axis.ticks = element_line(color = "black", linewidth = 0.5),

  # Sin leyenda (redundante con facetas)
  legend.position = "none",

  # Márgenes
  plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Normal Q-Q Plots: Sleep Quality by BMI Category",
  subtitle = "Assessment of normality assumption for each BMI group",
  x = "Theoretical Quantiles",

```

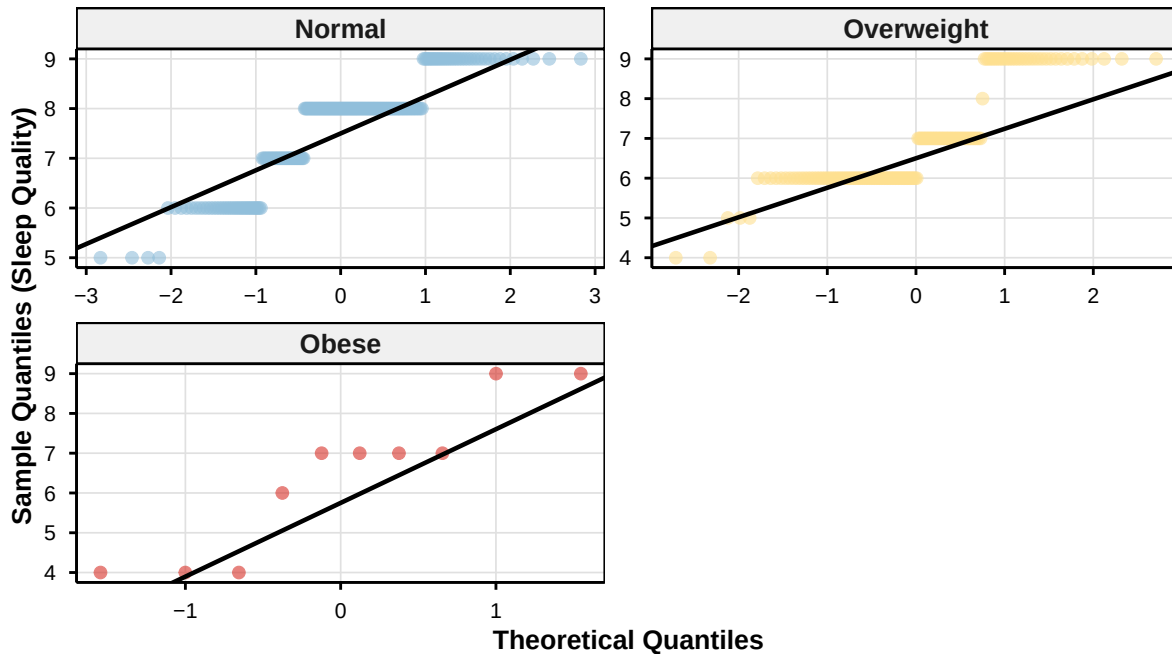
```

y = "Sample Quantiles (Sleep Quality)",
caption = "Colorblind-friendly palette | Black line represents theoretical normal distribution"
)

```

Normal Q-Q Plots: Sleep Quality by BMI Category

Assessment of normality assumption for each BMI group



Colorblind-friendly palette | Black line represents theoretical normal distribution

```

# Homogeneidad de varianzas
cat("\n Verificando homogeneidad de varianzas (Levene Test) \n")

```

```

##
## Verificando homogeneidad de varianzas (Levene Test)

```

```

library(car)

```

```

## S'està carregant el paquet requerit: carData

```

```

##
## S'està adjuntant el paquet: 'car'

```

```

## L'objecte següent està emmascarat per 'package:psych':
##
##   logit

```

```

## L'objecte següent està emmascarat per 'package:purrr':
##
##   some

```

```

## L'objecte següent està emmascarat per 'package:dplyr':
##
##      recode

print(leveneTest(Quality.of.Sleep ~ BMI.Category, data = df_1))

## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.

## Levene's Test for Homogeneity of Variance (center = median)
##      Df F value   Pr(>F)
## group  2  5.7635 0.003428 **
##      371
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

#TEST DE KRUSKAL-WALLIS

cat("\n Test de Kruskal-Wallis \n")

##
## Test de Kruskal-Wallis

kruskal_result <- kruskal.test(Quality.of.Sleep ~ BMI.Category, data = df_1)
print(kruskal_result)

##
## Kruskal-Wallis rank sum test
##
## data:  Quality.of.Sleep by BMI.Category
## Kruskal-Wallis chi-squared = 37.895, df = 2, p-value = 5.904e-09

# Post-hoc si es significativo
if (kruskal_result$p.value < 0.05) {
  cat("\n Post-hoc Test de Dunn (ajuste FDR - Benjamini-Hochberg) \n")
  library(FSA)
  dunn_result <- dunnTest(Quality.of.Sleep ~ BMI.Category, data = df_1, method = "bh") # FDR
  print(dunn_result)
}

##
## Post-hoc Test de Dunn (ajuste FDR - Benjamini-Hochberg)

## Registered S3 methods overwritten by 'FSA':
##      method      from
## confint.boot car
## hist.boot      car

## ## FSA v0.10.0. See citation('FSA') if used in publication.
## ## Run fishR() for related website and fishR('IFAR') for related book.

```

```
##
## S'està adjuntant el paquet: 'FSA'

## L'objecte següent està emmascarat per 'package:car':
##
##      bootCase

## L'objecte següent està emmascarat per 'package:psych':
##
##      headtail

## Warning: BMI.Category was coerced to a factor.

## Dunn (1964) Kruskal-Wallis multiple comparison

##      p-values adjusted with the Benjamini-Hochberg method.

##           Comparison           Z      P.unadj      P.adj
## 1      Normal - Obese  2.3140630 2.066427e-02 3.099640e-02
## 2 Normal - Overweight  5.9632587 2.472565e-09 7.417694e-09
## 3   Obese - Overweight -0.3433866 7.313076e-01 7.313076e-01

# Paleta accesible para daltonismo (misma que BMI anterior)
bmi_colors <- c(
  "Underweight" = "#4575B4", # Azul oscuro
  "Normal"      = "#91BFDB", # Azul claro
  "Overweight"  = "#FEE090", # Amarillo
  "Obese"       = "#D73027"  # Naranja-rojizo
)

# Ordenar categorías BMI
df_1_ordered <- df_1 %>%
  mutate(BMI.Category = factor(BMI.Category,
                              levels = c("Underweight", "Normal", "Overweight", "Obese")))

# Gráfica
p2 <- ggplot(df_1_ordered, aes(x = BMI.Category, y = Quality.of.Sleep, fill = BMI.Category)) +
  # Violin plots con bordes definidos
  geom_violin(trim = FALSE,
              alpha = 0.6,
              color = "black",
              linewidth = 0.5,
              scale = "width") +
  # Boxplot superpuesto más pequeño
  geom_boxplot(width = 0.15,
              alpha = 0.8,
              color = "black",
              linewidth = 0.4,
              outlier.size = 1.5,
              outlier.alpha = 0.7) +
  # Escala de colores accesible
  scale_fill_manual(values = bmi_colors) +
```

```

# Escala Y continua
scale_y_continuous(breaks = seq(0, 10, 2),
                   limits = c(0, NA),
                   expand = expansion(mult = c(0, 0.05))) +
# Tema personalizado estilo Nature
theme_minimal(base_size = 11, base_family = "sans") +
theme(
  # Panel y fondo
  panel.background = element_rect(fill = "white", color = NA),
  panel.grid.major.y = element_line(color = "#E0E0E0", linewidth = 0.3),
  panel.grid.major.x = element_blank(),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

  # Títulos y texto
  plot.title = element_text(size = 14, face = "bold", hjust = 0,
                             margin = margin(b = 5)),
  plot.subtitle = element_text(size = 10, hjust = 0,
                                color = "grey30",
                                margin = margin(b = 10)),

  # Ejes
  axis.title = element_text(size = 10, face = "bold"),
  axis.text.y = element_text(size = 9, color = "black"),
  axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1,
                              size = 9, color = "black", face = "bold"),
  axis.line = element_line(color = "black", linewidth = 0.5),
  axis.ticks = element_line(color = "black", linewidth = 0.5),

  # Sin leyenda
  legend.position = "none",

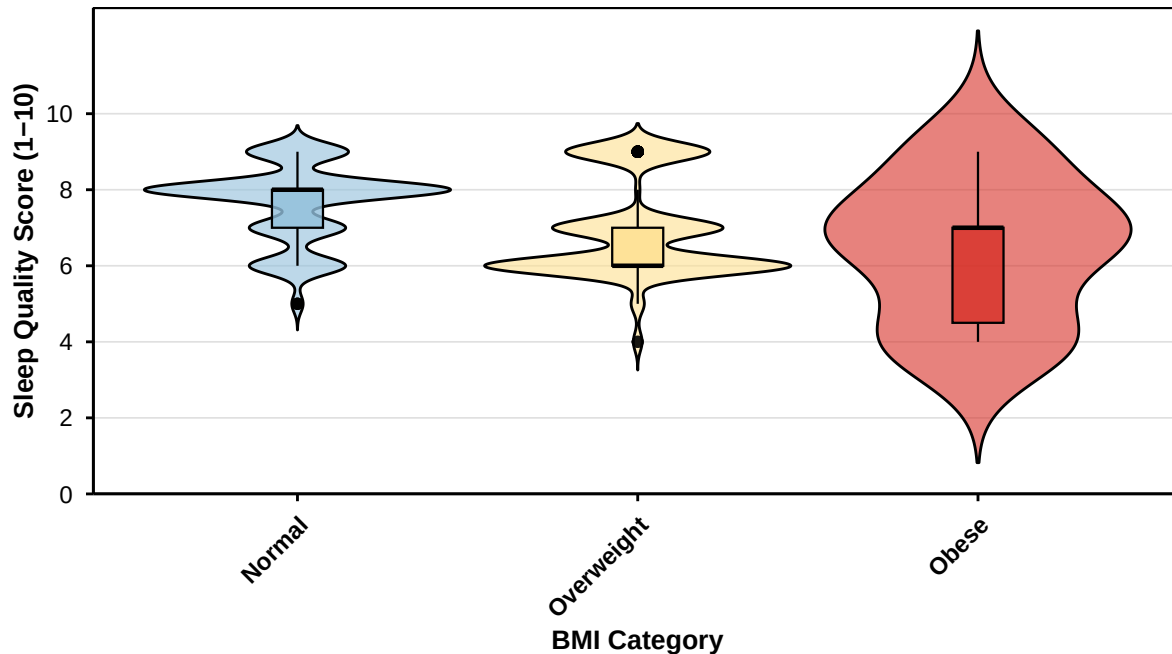
  # Márgenes
  plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Sleep Quality Distribution by BMI Category",
  subtitle = "Violin plots showing probability density with embedded box plots",
  x = "BMI Category",
  y = "Sleep Quality Score (1-10)",
  caption = "Colorblind-friendly palette | Box plots show median, IQR, and outliers"
)

# Mostrar la gráfica
print(p2)

```

Sleep Quality Distribution by BMI Category

Violin plots showing probability density with embedded box plots



Colorblind-friendly palette | Box plots show median, IQR, and outliers

Estos resultados sugieren que la calidad del sueño se ve afectada principalmente al pasar de un IMC normal a valores superiores, especialmente en el caso del sobrepeso. La diferencia entre sobrepeso y obesidad no es estadísticamente significativa, lo que indica que la disminución en la calidad del sueño se produce principalmente en la transición de un peso normal a exceso de peso (overweight y obese).

3.¿Qué variables explican mejor la calidad del sueño?

```
# Preparar datos para regresión
# Seleccionar variables numéricas relevantes
#vars_for_regression <- df_1 %>%
#  select(Age, Sleep.Duration, Physical.Activity.Level, Stress.Level,
#         Heart.Rate, Daily.Steps, Quality.of.Sleep) %>%
#  na.omit()

#Quitamos sleep duration ya que esta muy relacionada con quality of sleep
vars_for_regression <- df_1 %>%
  select(Age, Physical.Activity.Level, Stress.Level,
         Heart.Rate, Daily.Steps, Quality.of.Sleep) %>%
  na.omit()
# Separar X e y
X <- as.matrix(vars_for_regression %>% select(-Quality.of.Sleep))
y <- vars_for_regression$Quality.of.Sleep

# Dividir en train y test
set.seed(123)
```

```
train_idx <- sample(1:nrow(X), 0.7 * nrow(X))
X_train <- X[train_idx, ]
y_train <- y[train_idx]
X_test <- X[-train_idx, ]
y_test <- y[-train_idx]
```

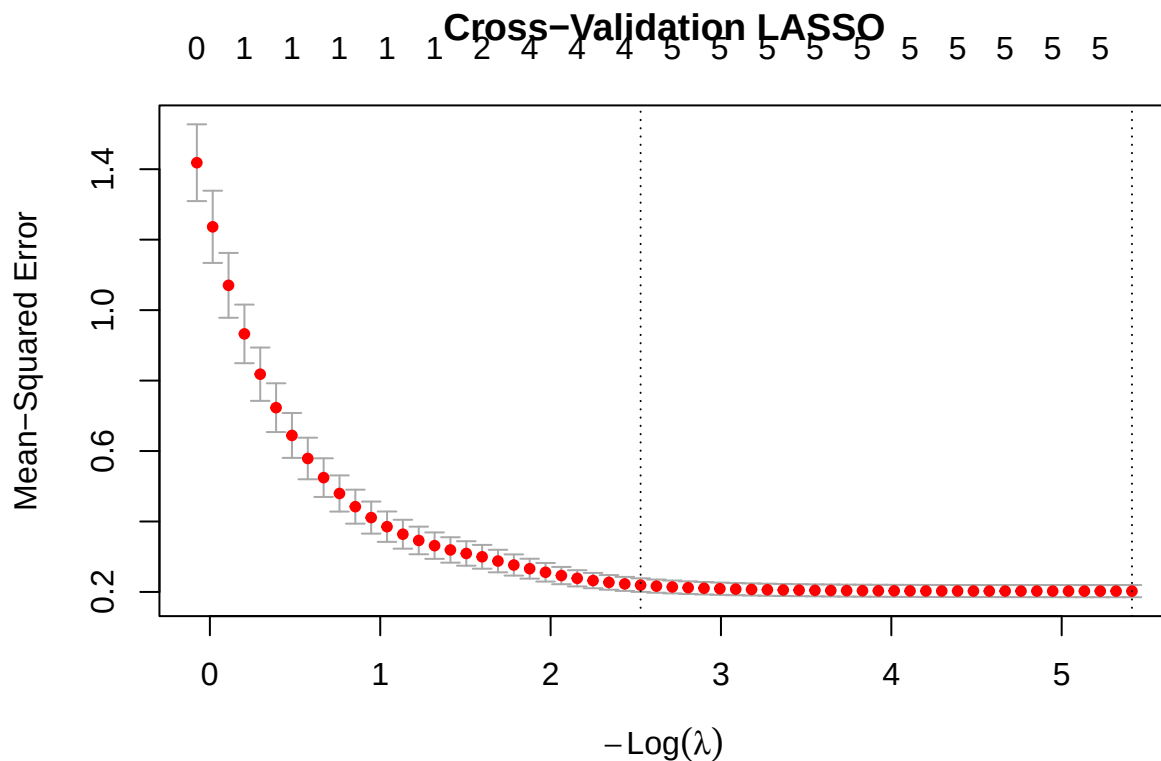
```
# Regresión Lasso (alpha = 1)
cat("\n LASSO REGRESSION \n")
```

```
##
## LASSO REGRESSION
```

```
lasso_model <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 10)
cat("Lambda óptimo:", lasso_model$lambda.min, "\n")
```

```
## Lambda óptimo: 0.00446074
```

```
plot(lasso_model, main = "Cross-Validation LASSO")
```



```
# Coeficientes Lasso
lasso_coef <- coef(lasso_model, s = "lambda.min")
cat("\nCoeficientes LASSO:\n")
```

```
##
## Coeficientes LASSO:
```

```

print(lasso_coef)

## 6 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.min
## (Intercept)    1.159047e+01
## Age           1.110404e-02
## Physical.Activity.Level 7.963536e-03
## Stress.Level    -5.378902e-01
## Heart.Rate      -3.620767e-02
## Daily.Steps     3.230198e-05

# Predicción y R-squared
pred_lasso <- predict(lasso_model, s = "lambda.min", newx = X_test)
r2_lasso <- 1 - sum((y_test - pred_lasso)^2) / sum((y_test - mean(y_test))^2)
cat("R² LASSO:", r2_lasso, "\n")

## R² LASSO: 0.8240171

# Regresión Ridge (alpha = 0)
cat("\n RIDGE REGRESSION \n")

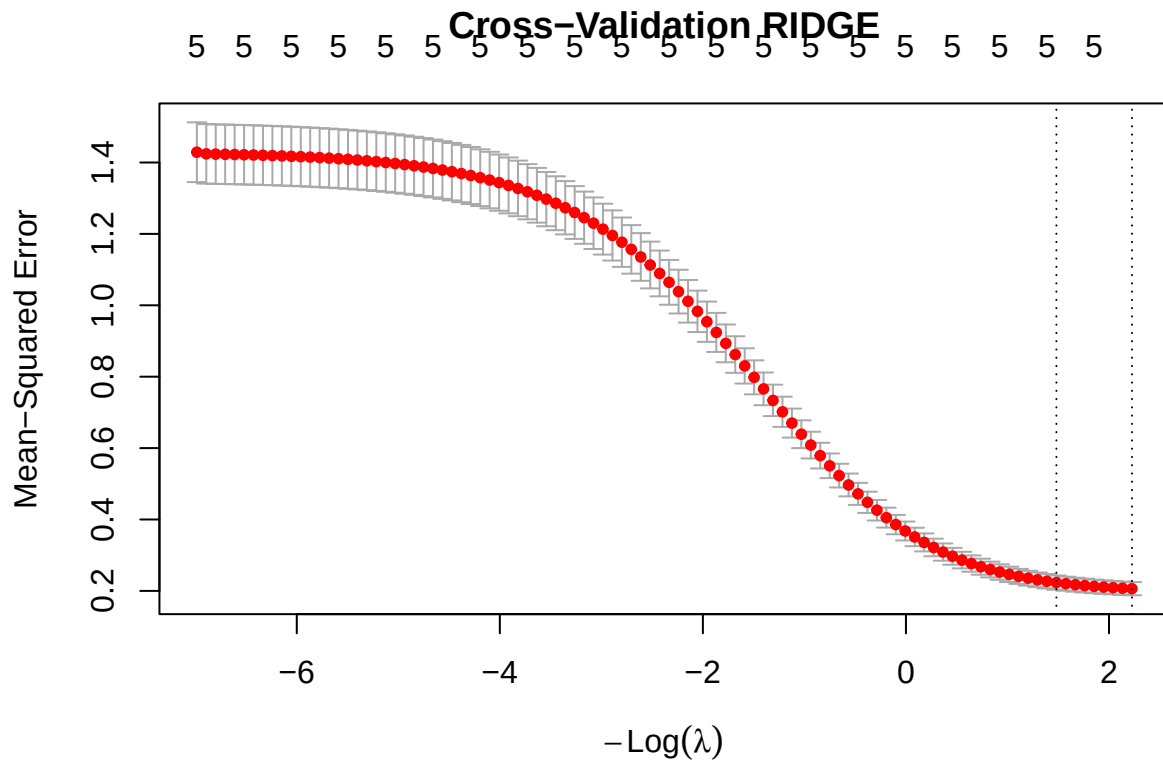
##
## RIDGE REGRESSION

ridge_model <- cv.glmnet(X_train, y_train, alpha = 0, nfolds = 10)
cat("Lambda óptimo:", ridge_model$lambda.min, "\n")

## Lambda óptimo: 0.1079556

plot(ridge_model, main = "Cross-Validation RIDGE")

```

```
# Coeficientes Ridge
ridge_coef <- coef(ridge_model, s = "lambda.min")
cat("\nCoeficientes RIDGE:\n")
```

```
##
## Coeficientes RIDGE:
```

```
print(ridge_coef)
```

```
## 6 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.min
## (Intercept)    1.252993e+01
## Age            1.502031e-02
## Physical.Activity.Level 9.989880e-03
## Stress.Level    -4.529107e-01
## Heart.Rate      -5.647177e-02
## Daily.Steps     -5.585647e-06
```

```
# Predicción y R-squared
pred_ride <- predict(ridge_model, s = "lambda.min", newx = X_test)
r2_ride <- 1 - sum((y_test - pred_ride)^2) / sum((y_test - mean(y_test))^2)
cat("R² RIDGE:", r2_ride, "\n")
```

```
## R² RIDGE: 0.827431
```

```

# Comparación visual de coeficientes
coef_comparison <- data.frame(
  Variable = rownames(lasso_coef)[-1],
  Lasso = as.vector(lasso_coef)[-1],
  Ridge = as.vector(ridge_coef)[-1]
)

# Paleta accesible para daltonismo
method_colors <- c(
  "Lasso" = "#0173B2", # Azul
  "Ridge" = "#DE8F05"  # Naranja
)

# Preparar datos
coef_data <- coef_comparison %>%
  pivot_longer(cols = c(Lasso, Ridge),
    names_to = "Method",
    values_to = "Coefficient") %>%
  # Ordenar variables por magnitud promedio de coeficientes
  group_by(Variable) %>%
  mutate(avg_coef = mean(abs(Coefficient))) %>%
  ungroup() %>%
  arrange(avg_coef) %>%
  mutate(Variable = factor(Variable, levels = unique(Variable)))

# Gráfica en estilo Nature
p5 <- ggplot(coef_data, aes(x = Variable, y = Coefficient, fill = Method)) +
  # Barras agrupadas con bordes
  geom_bar(stat = "identity",
    position = position_dodge(width = 0.8),
    alpha = 0.85,
    color = "black",
    linewidth = 0.3,
    width = 0.75) +
  # Línea de referencia en cero
  geom_hline(yintercept = 0,
    color = "black",
    linewidth = 0.5,
    linetype = "solid") +
  # Escala de colores accesible
  scale_fill_manual(values = method_colors,
    name = "Regularization\nMethod") +
  # Coordenadas volteadas
  coord_flip() +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    # Panel y fondo
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major.x = element_line(color = "#E0E0E0", linewidth = 0.3),
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

```

```

# Títulos y texto
plot.title = element_text(size = 14, face = "bold", hjust = 0,
                           margin = margin(b = 5)),
plot.subtitle = element_text(size = 10, hjust = 0,
                              color = "grey30",
                              margin = margin(b = 10)),

# Ejes
axis.title = element_text(size = 10, face = "bold"),
axis.text.x = element_text(size = 9, color = "black"),
axis.text.y = element_text(size = 9, color = "black", hjust = 1),
axis.line = element_line(color = "black", linewidth = 0.5),
axis.ticks = element_line(color = "black", linewidth = 0.5),

# Leyenda
legend.position = "top",
legend.justification = "left",
legend.title = element_text(size = 10, face = "bold"),
legend.text = element_text(size = 9),
legend.background = element_rect(fill = "white", color = "black", linewidth = 0.3),
legend.key.size = unit(0.7, "cm"),
legend.margin = margin(5, 5, 5, 5),
legend.box.spacing = unit(0.3, "cm"),

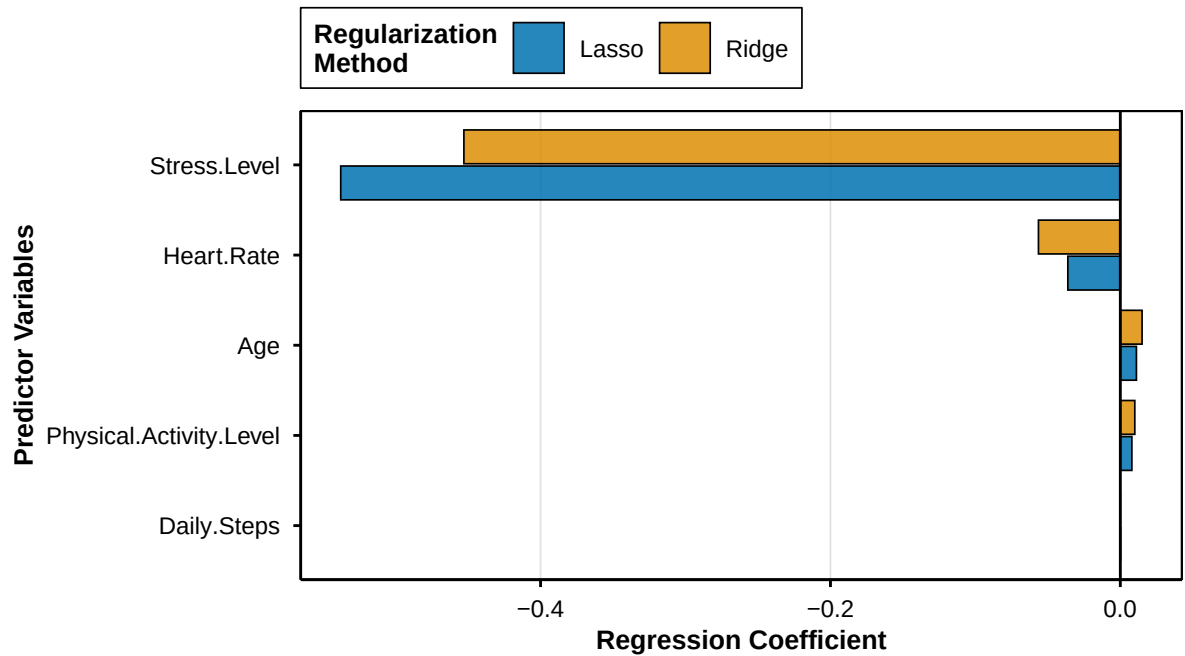
# Márgenes
plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Regularization Coefficient Comparison: LASSO vs Ridge",
  subtitle = "Feature importance and shrinkage patterns in penalized regression models",
  x = "Predictor Variables",
  y = "Regression Coefficient",
  caption = "Colorblind-friendly palette | Variables ordered by average coefficient magnitude"
)

# Mostrar la gráfica
print(p5)

```

Regularization Coefficient Comparison: LASSO vs Ridge

Feature importance and shrinkage patterns in penalized regression models



Colorblind-friendly palette | Variables ordered by average coefficient magnitude

Podemos observar que las variables que mejor explican la calidad del sueño son el nivel de estrés y la duración del sueño. La actividad física, el pulso, los pasos diarios y la edad, prácticamente no influyen.

4. ¿Tener una alteración del sueño influye significativamente en la calidad del sueño?

```
# Verificar si la variable es numérica o categórica
cat("\nTipo de variable Stress Level:\n")
```

```
##
## Tipo de variable Stress Level:
```

```
str(df_1$Stress.Level)
```

```
## int [1:374] 6 8 8 8 8 8 7 6 6 6 ...
```

```
# Correlación de Pearson
cor_test <- cor.test(df_1$Stress.Level, df_1$Quality.of.Sleep,
                     method = "pearson")
cat("\nCorrelación de Pearson:\n")
```

```
##
## Correlación de Pearson:
```

```
print(cor_test)
```

```
##  
## Pearson's product-moment correlation  
##  
## data: df_1$Stress.Level and df_1$Quality.of.Sleep  
## t = -39.535, df = 372, p-value < 2.2e-16  
## alternative hypothesis: true correlation is not equal to 0  
## 95 percent confidence interval:  
## -0.9166186 -0.8773022  
## sample estimates:  
## cor  
## -0.898752
```

```
# Correlación de Spearman (alternativa no paramétrica)  
cor_test_spearman <- cor.test(df_1$Stress.Level, df_1$Quality.of.Sleep,  
                             method = "spearman")
```

```
## Warning in cor.test.default(df_1$Stress.Level, df_1$Quality.of.Sleep, method =  
## "spearman"): Cannot compute exact p-value with ties
```

```
cat("\nCorrelación de Spearman:\n")
```

```
##  
## Correlación de Spearman:
```

```
print(cor_test_spearman)
```

```
##  
## Spearman's rank correlation rho  
##  
## data: df_1$Stress.Level and df_1$Quality.of.Sleep  
## S = 16636896, p-value < 2.2e-16  
## alternative hypothesis: true rho is not equal to 0  
## sample estimates:  
## rho  
## -0.908147
```

```
# Colores accesibles para daltonismo  
color_points <- "#0173B2"      # Azul para puntos  
color_regression <- "#DE8F05"  # Naranja para línea de regresión  
color_ci <- "#FDB462"         # Naranja claro para intervalo de confianza  
  
# Calcular estadísticas de correlación  
correlation <- cor(df_1$Stress.Level, df_1$Quality.of.Sleep, use = "complete.obs")  
model <- lm(Quality.of.Sleep ~ Stress.Level, data = df_1)  
r_squared <- summary(model)$r.squared  
p_value <- summary(model)$coefficients[2, 4]  
  
# Texto de estadísticas
```

```

stats_text <- paste0(
  "r = ", round(correlation, 3),
  " (R² = ", round(r_squared, 3), ")",
  ifelse(p_value < 0.001, "\np < 0.001", paste0("\np = ", round(p_value, 3)))
)

# Gráfica en estilo Nature
p3 <- ggplot(df_1, aes(x = Stress.Level, y = Quality.of.Sleep)) +
  # Intervalo de confianza primero (detrás)
  geom_smooth(method = "lm",
    se = TRUE,
    color = color_regression,
    fill = color_ci,
    alpha = 0.3,
    linewidth = 1.2) +
  # Puntos encima
  geom_point(color = color_points,
    alpha = 0.5,
    size = 2.5,
    shape = 16) +
  # Añadir estadísticas en el gráfico
  annotate("text",
    x = max(df_1$Stress.Level, na.rm = TRUE) - 1,
    y = max(df_1$Quality.of.Sleep, na.rm = TRUE) - 0.5,
    label = stats_text,
    hjust = 1,
    vjust = 1,
    size = 3.5,
    fontface = "bold",
    color = "black") +
  # Escalas
  scale_x_continuous(breaks = seq(0, 10, 2)) +
  scale_y_continuous(breaks = seq(0, 10, 2)) +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    # Panel y fondo
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

    # Títulos y texto
    plot.title = element_text(size = 14, face = "bold", hjust = 0,
      margin = margin(b = 5)),
    plot.subtitle = element_text(size = 10, hjust = 0,
      color = "grey30",
      margin = margin(b = 10)),

    # Ejes
    axis.title = element_text(size = 10, face = "bold"),
    axis.text = element_text(size = 9, color = "black"),
    axis.line = element_line(color = "black", linewidth = 0.5),

```

```

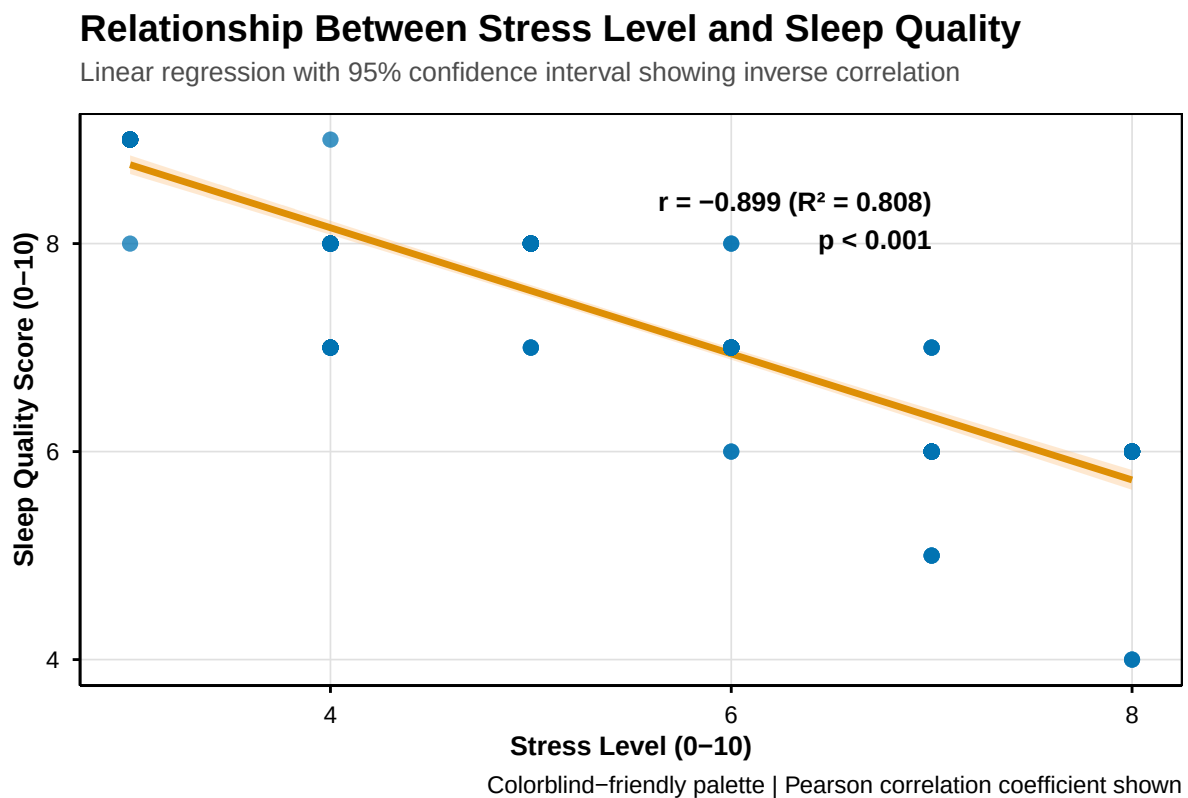
axis.ticks = element_line(color = "black", linewidth = 0.5),

# Márgenes
plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "Relationship Between Stress Level and Sleep Quality",
  subtitle = "Linear regression with 95% confidence interval showing inverse correlation",
  x = "Stress Level (0-10)",
  y = "Sleep Quality Score (0-10)",
  caption = "Colorblind-friendly palette | Pearson correlation coefficient shown"
)

# Mostrar la gráfica
print(p3)

## `geom_smooth()` using formula = 'y ~ x'

```



Los resultados muestran una relación negativa clara (en Pearson de -0.89 y en Spearman de -0.90) entre el nivel de estrés y la calidad del sueño. A medida que aumenta el nivel de estrés, la calidad del sueño tiende a disminuir. Esta tendencia queda reflejada en la línea de regresión descendente del gráfico. Obviamente, nos sale un p-valor muy significativo en ambos casos.

5. ¿Podemos predecir si una persona sufre insomnio o apnea?

```
# Explorar la variable Sleep.Disorder
cat("\nCategorías de trastornos del sueño:\n")

##
## Categorías de trastornos del sueño:

table(df_1$Sleep.Disorder)

##
##      Insomnia      None Sleep Apnea
##          77          219          78

# Crear variable binaria: 1 si tiene trastorno (Insomnia o Sleep Apnea), 0 si no tiene trastorno.
df_1$Has.Disorder <- ifelse(df_1$Sleep.Disorder %in% c("Insomnia", "Sleep Apnea"), 1, 0)

cat("\nDistribución de trastornos:\n")

##
## Distribución de trastornos:

table(df_1$Has.Disorder)

##
##    0    1
## 219 155

# Seleccionar variables predictoras
# Nota: Algunas variables pueden necesitar transformación
predictors <- df_1 %>%
  select(Physical.Activity.Level, Stress.Level, BMI.Category) %>%
  mutate(BMI.Category = as.factor(BMI.Category))

# Crear variables dummy para BMI.Category
predictors_encoded <- model.matrix(~ Physical.Activity.Level + Stress.Level + BMI.Category - 1,
                                   data = predictors)

# Preparar datos completos
data_logit <- data.frame(Has.Disorder = df_1$Has.Disorder, predictors_encoded)

# Dividir en train y test
set.seed(123)
train_index <- createDataPartition(data_logit$Has.Disorder, p = 0.7, list = FALSE)
train_data <- data_logit[train_index, ]
test_data <- data_logit[-train_index, ]

# Modelo de regresión logística
logit_model <- glm(Has.Disorder ~ ., data = train_data, family = binomial)
cat("\nResumen del modelo de regresión logística:\n")
```



```
##
## Resumen del modelo de regresión logística:

print(summary(logit_model))

##
## Call:
## glm(formula = Has.Disorder ~ ., family = binomial, data = train_data)
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.13300    1.09229   0.122   0.903
## Physical.Activity.Level 0.02057    0.01167   1.762   0.078 .
## Stress.Level     0.08077    0.13490   0.599   0.549
## BMI.CategoryNormal -4.61254    0.46732  -9.870 <2e-16 ***
## BMI.CategoryObese  15.85124  1485.81477   0.011   0.991
## BMI.CategoryOverweight      NA         NA      NA      NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 353.61  on 261  degrees of freedom
## Residual deviance: 150.59  on 257  degrees of freedom
## AIC: 160.59
##
## Number of Fisher Scoring iterations: 16
```

```
# Predicciones
pred_prob <- predict(logit_model, test_data, type = "response")
pred_class <- ifelse(pred_prob > 0.5, 1, 0)

# Matriz de confusión
conf_matrix <- confusionMatrix(as.factor(pred_class), as.factor(test_data$Has.Disorder))
cat("\nMatriz de confusión:\n")
```

```
##
## Matriz de confusión:
```

```
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction  0  1
##           0 59  7
##           1  4 42
##
##              Accuracy : 0.9018
##              95% CI : (0.8311, 0.9499)
##      No Information Rate : 0.5625
##      P-Value [Acc > NIR] : 3.952e-15
```

```
##
##           Kappa : 0.7991
##
## Mcnemar's Test P-Value : 0.5465
##
##           Sensitivity : 0.9365
##           Specificity : 0.8571
##           Pos Pred Value : 0.8939
##           Neg Pred Value : 0.9130
##           Prevalence : 0.5625
##           Detection Rate : 0.5268
##           Detection Prevalence : 0.5893
##           Balanced Accuracy : 0.8968
##
##           'Positive' Class : 0
##
```

```
# Colores accesibles para daltonismo
color_roc <- "#0173B2"      # Azul para curva ROC
color_diagonal <- "#DE8F05" # Naranja para línea de referencia
color_ci <- "#91BFDB"       # Azul claro para IC

# Calcular curva ROC
if (require(pROC, quietly = TRUE)) {
  roc_curve <- roc(test_data$Has.Disorder, pred_prob)
  auc_value <- auc(roc_curve)
  ci_auc <- ci.auc(roc_curve)

  cat("\nAUC:", auc_value, "\n")
  cat("95% CI:", ci_auc[1], "-", ci_auc[3], "\n")

  # Extraer coordenadas de la curva ROC
  roc_data <- data.frame(
    specificity = roc_curve$specificities,
    sensitivity = roc_curve$sensitivities,
    thresholds = roc_curve$thresholds
  )

  # Encontrar punto óptimo (Youden's index)
  youden_index <- which.max(roc_curve$sensitivities + roc_curve$specificities - 1)
  optimal_point <- data.frame(
    specificity = roc_curve$specificities[youden_index],
    sensitivity = roc_curve$sensitivities[youden_index],
    threshold = roc_curve$thresholds[youden_index]
  )

  # Texto de AUC
  auc_text <- paste0(
    "AUC = ", round(auc_value, 3),
    "\n95% CI: [", round(ci_auc[1], 3), " - ", round(ci_auc[3], 3), "]"
  )

  # Gráfica en estilo Nature
  p_roc <- ggplot(roc_data, aes(x = 1 - specificity, y = sensitivity)) +
```

```

# Área bajo la curva (sombreada)
geom_ribbon(aes(ymin = 0, ymax = sensitivity),
            fill = color_ci,
            alpha = 0.3) +
# Curva ROC
geom_line(color = color_roc,
          linewidth = 1.5) +
# Línea de referencia diagonal (clasificador aleatorio)
geom_abline(intercept = 0, slope = 1,
            color = color_diagonal,
            linetype = "dashed",
            linewidth = 1) +
# Punto óptimo
#geom_point(data = optimal_point,
#           aes(x = 1 - specificity, y = sensitivity),
#           color = "#CC0000",
#           size = 4,
#           shape = 18) +
# Anotación del punto óptimo
#annotate("text",
#         x = 1 - optimal_point$specificity + 0.1,
#         y = optimal_point$sensitivity - 0.05,
#         label = paste0("Optimal threshold\n",
#                        "Sens: ", round(optimal_point$sensitivity, 2),
#                        " | Spec: ", round(optimal_point$specificity, 2)),
#         hjust = 0,
#         size = 3,
#         color = "black") +
# Texto de AUC
annotate("text",
        x = 0.6,
        y = 0.2,
        label = auc_text,
        hjust = 0,
        size = 4,
        fontface = "bold",
        color = "black") +
# Escalas
scale_x_continuous(breaks = seq(0, 1, 0.2),
                  expand = c(0, 0)) +
scale_y_continuous(breaks = seq(0, 1, 0.2),
                  expand = c(0, 0)) +
# Tema personalizado estilo Nature
theme_minimal(base_size = 11, base_family = "sans") +
theme(
  # Panel y fondo
  panel.background = element_rect(fill = "white", color = NA),
  panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
  panel.grid.minor = element_blank(),
  panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),

  # Títulos y texto
  plot.title = element_text(size = 14, face = "bold", hjust = 0,

```

```

        margin = margin(b = 5)),
plot.subtitle = element_text(size = 10, hjust = 0,
                             color = "grey30",
                             margin = margin(b = 10)),

# Ejes
axis.title = element_text(size = 10, face = "bold"),
axis.text = element_text(size = 9, color = "black"),
axis.line = element_line(color = "black", linewidth = 0.5),
axis.ticks = element_line(color = "black", linewidth = 0.5),

# Aspecto cuadrado
aspect.ratio = 1,

# Márgenes
plot.margin = margin(15, 15, 15, 15)
) +
labs(
  title = "ROC Curve: Sleep Disorder Prediction Model",
  subtitle = "Receiver Operating Characteristic with optimal threshold identification",
  x = "False Positive Rate (1 - Specificity)",
  y = "True Positive Rate (Sensitivity)",
  caption = "Colorblind-friendly palette | Diamond indicates optimal Youden's index threshold"
)

# Mostrar la gráfica
print(p_roc)
} else {
  message("El paquete pROC no está instalado. Instálalo con: install.packages('pROC')")
}

```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## S'està adjuntant el paquet: 'pROC'
```

```
## Els següents objectes estan emmascarats des de 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

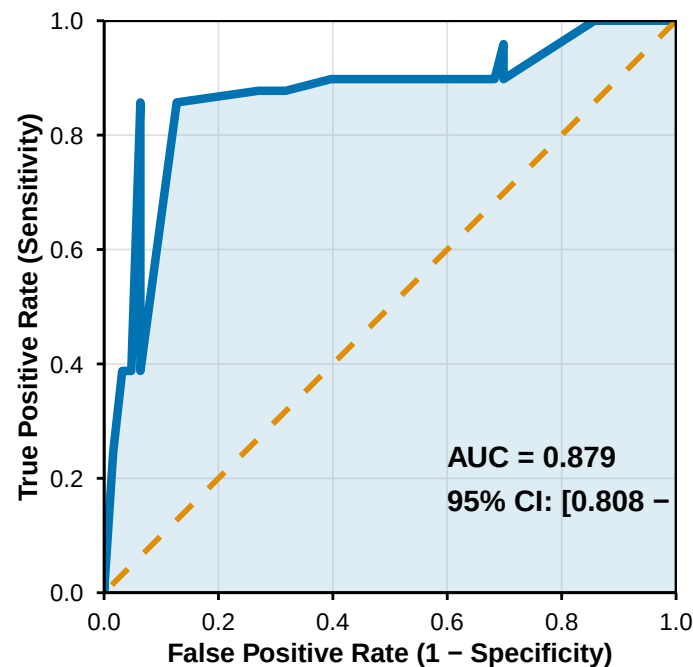
```
##
```

```
## AUC: 0.8794947
```

```
## 95% CI: 0.8075295 - 0.9514598
```

ROC Curve: Sleep Disorder Prediction Model

Receiver Operating Characteristic with optimal threshold identification



Colorblind-friendly palette | Diamond indicates optimal Youden's index threshold

CAMBIAR #En base a los resultados obtenidos, podemos concluir que: #-El modelo de regresión logística tiene una buena capacidad para predecir (en base a las variables de actividad física, nivel de estrés y BMI) si un paciente padece o no de trastorno del sueño (con una sensibilidad del 93,6% y especificidad de 85,7%). -El AUC nos sale de 0.88, la cual cosa confirma que el modelo es capaz de discriminar entre las personas con o sin trastorno del sueño. -Por último, pero no menos importante, no hay evidencia de que el modelo esté sobreajustado, factor muy importante de cara a poder extrapolar su uso en otros conjuntos de datos.

6. ¿Existen patrones latentes que agrupan a los individuos?(Tambien he eliminado quality of sleep)

```
# Instalar paquetes si no los tienes
# install.packages(c("dplyr", "ggplot2", "factoextra", "cluster"))

# Seleccionamos variables numéricas relevantes
df_1_num <- df_1 %>%
  select(Age, Sleep.Duration, Physical.Activity.Level,
         Stress.Level, Heart.Rate, Daily.Steps)

# Estandarizamos.
df_1_scaled <- scale(df_1_num)

library(factoextra)
library(patchwork) # Para combinar gráficas
```

```

# Colores accesibles para daltonismo
color_main <- "#0173B2"      # Azul para curvas/barras
color_optimal <- "#DE8F05"   # Naranja para punto óptimo
color_line <- "#CC0000"      # Rojo para líneas de referencia

# MÉTODO DEL CODO (WSS - Within Sum of Squares)

# Calcular WSS para diferentes números de clusters
set.seed(123)
wss_values <- sapply(1:10, function(k) {
  kmeans(df_1_scaled, centers = k, nstart = 25)$tot.withinss
})

wss_data <- data.frame(
  k = 1:10,
  wss = wss_values
)

# Identificar el codo (método simple: mayor cambio de pendiente)
# Calcular segunda derivada
first_diff <- diff(wss_values)
second_diff <- diff(first_diff)
optimal_k_wss <- which.max(abs(second_diff)) + 1

# Gráfica Método del Codo
p_wss <- ggplot(wss_data, aes(x = k, y = wss)) +
  # Línea principal
  geom_line(color = color_main,
            linewidth = 1.2) +
  # Puntos
  geom_point(color = color_main,
             size = 3,
             shape = 16) +
  # Escalas
  scale_x_continuous(breaks = 1:10) +
  scale_y_continuous(labels = scales::comma) +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major = element_line(color = "#E0E0E0", linewidth = 0.3),
    panel.grid.minor = element_blank(),
    panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
    plot.title = element_text(size = 12, face = "bold", hjust = 0,
                              margin = margin(b = 5)),
    plot.subtitle = element_text(size = 9, hjust = 0, color = "grey30",
                                 margin = margin(b = 5)),
    axis.title = element_text(size = 10, face = "bold"),
    axis.text = element_text(size = 9, color = "black"),
    axis.line = element_line(color = "black", linewidth = 0.5),
    axis.ticks = element_line(color = "black", linewidth = 0.5),
    plot.margin = margin(10, 10, 10, 10)
  ) +

```

```

labs(
  title = "Elbow Method (WSS)",
  subtitle = "Within-cluster sum of squares by number of clusters",
  x = "Number of Clusters (k)",
  y = "Total Within-Cluster SS"
)

# MÉTODO DEL SILHOUETTE

# Calcular Silhouette para diferentes números de clusters
silhouette_values <- sapply(2:10, function(k) {
  km <- kmeans(df_1_scaled, centers = k, nstart = 25)
  ss <- cluster::silhouette(km$cluster, dist(df_1_scaled))
  mean(ss[, 3])
})

silhouette_data <- data.frame(
  k = 2:10,
  silhouette = silhouette_values
)

# Identificar el k óptimo (máximo silhouette)
optimal_k_sil <- silhouette_data$k[which.max(silhouette_data$silhouette)]

# Gráfica Método del Silhouette
p_sil <- ggplot(silhouette_data, aes(x = k, y = silhouette)) +
  # Barras
  geom_col(fill = color_main,
    alpha = 0.85,
    color = "black",
    linewidth = 0.3,
    width = 0.7) +
  # Etiquetas de valores
  geom_text(aes(label = round(silhouette, 3)),
    vjust = -0.5,
    size = 3,
    fontface = "bold",
    color = "black") +
  # Línea horizontal de referencia en el máximo
  geom_hline(yintercept = max(silhouette_values),
    color = color_optimal,
    linetype = "dashed",
    linewidth = 0.8) +
  # Escalas
  scale_x_continuous(breaks = 2:10) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15)),
    limits = c(0, NA)) +
  # Tema personalizado estilo Nature
  theme_minimal(base_size = 11, base_family = "sans") +
  theme(
    panel.background = element_rect(fill = "white", color = NA),
    panel.grid.major.y = element_line(color = "#E0E0E0", linewidth = 0.3),
    panel.grid.major.x = element_blank(),

```

```

panel.grid.minor = element_blank(),
panel.border = element_rect(color = "black", fill = NA, linewidth = 0.5),
plot.title = element_text(size = 12, face = "bold", hjust = 0,
                           margin = margin(b = 5)),
plot.subtitle = element_text(size = 9, hjust = 0, color = "grey30",
                              margin = margin(b = 5)),
axis.title = element_text(size = 10, face = "bold"),
axis.text = element_text(size = 9, color = "black"),
axis.line = element_line(color = "black", linewidth = 0.5),
axis.ticks = element_line(color = "black", linewidth = 0.5),
plot.margin = margin(10, 10, 10, 10)
) +
labs(
  title = "Silhouette Method",
  subtitle = "Average silhouette width by number of clusters",
  x = "Number of Clusters (k)",
  y = "Average Silhouette Width"
)

# COMBINAR AMBAS GRÁFICAS

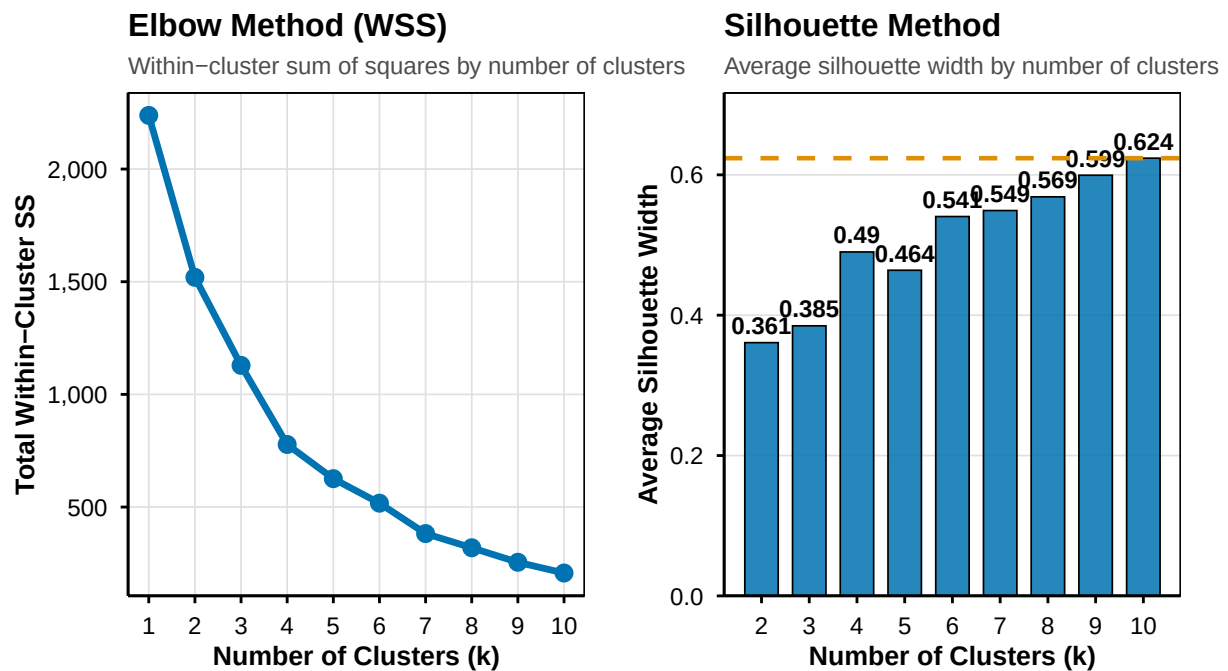
# Usar patchwork para combinar
combined_plot <- p_wss + p_sil +
  plot_annotation(
    title = "Optimal Number of Clusters: K-means Method Comparison",
    subtitle = "Elbow and Silhouette methods for determining optimal k",
    caption = "Colorblind-friendly palette | Orange indicates optimal k for each method",
    theme = theme(
      plot.title = element_text(size = 14, face = "bold", hjust = 0),
      plot.subtitle = element_text(size = 10, color = "grey30", hjust = 0),
      plot.caption = element_text(size = 8, hjust = 0)
    )
  )

# Mostrar la gráfica combinada
print(combined_plot)

```


Optimal Number of Clusters: K-means Method Comparison

Elbow and Silhouette methods for determining optimal k



Colorblind-friendly palette | Orange indicates optimal k for each method

```
# Imprimir resultados
cat("\n=== Resultados de Optimización ===\n")

##
## === Resultados de Optimización ===

cat("Método del Codo (WSS) sugiere k =", optimal_k_wss, "\n")

## Método del Codo (WSS) sugiere k = 2

cat("Método Silhouette sugiere k =", optimal_k_sil, "\n")

## Método Silhouette sugiere k = 10

# Aplicamos K-Means con K = 3.
set.seed(123)
k <- 3
kmeans_res <- kmeans(df_1_scaled, centers = k, nstart = 25)

# Añadimos el Clúster al dataFrame original.
df_1$Cluster <- as.factor(kmeans_res$cluster)

# Interpretamos los clústers con medias por grupo.
```

```
cluster_summary <- df_1 %>%
  group_by(Cluster) %>%
  summarise(
    Age = mean(Age, na.rm = TRUE),
    Sleep.Duration = mean(Sleep.Duration, na.rm = TRUE),
    #Quality.of.Sleep = mean(Quality.of.Sleep, na.rm = TRUE),
    Physical.Activity.Level = mean(Physical.Activity.Level, na.rm = TRUE),
    Stress.Level = mean(Stress.Level, na.rm = TRUE),
    Heart.Rate = mean(Heart.Rate, na.rm = TRUE),
    Daily.Steps = mean(Daily.Steps, na.rm = TRUE)
  )

print(cluster_summary)
```

```
## # A tibble: 3 x 7
##   Cluster Age Sleep.Duration Physical.Activity.Level Stress.Level Heart.Rate
##   <fct>   <dbl>         <dbl>          <dbl>         <dbl>     <dbl>
## 1 1      43.0           7.60           63.0           4.28      68.0
## 2 2      38.0           6.36           40.0           7.15      73.8
## 3 3      48.5           6.07           88.2           8         75
## # i 1 more variable: Daily.Steps <dbl>
```

```
# Renombramos los clústers.
library(dplyr)

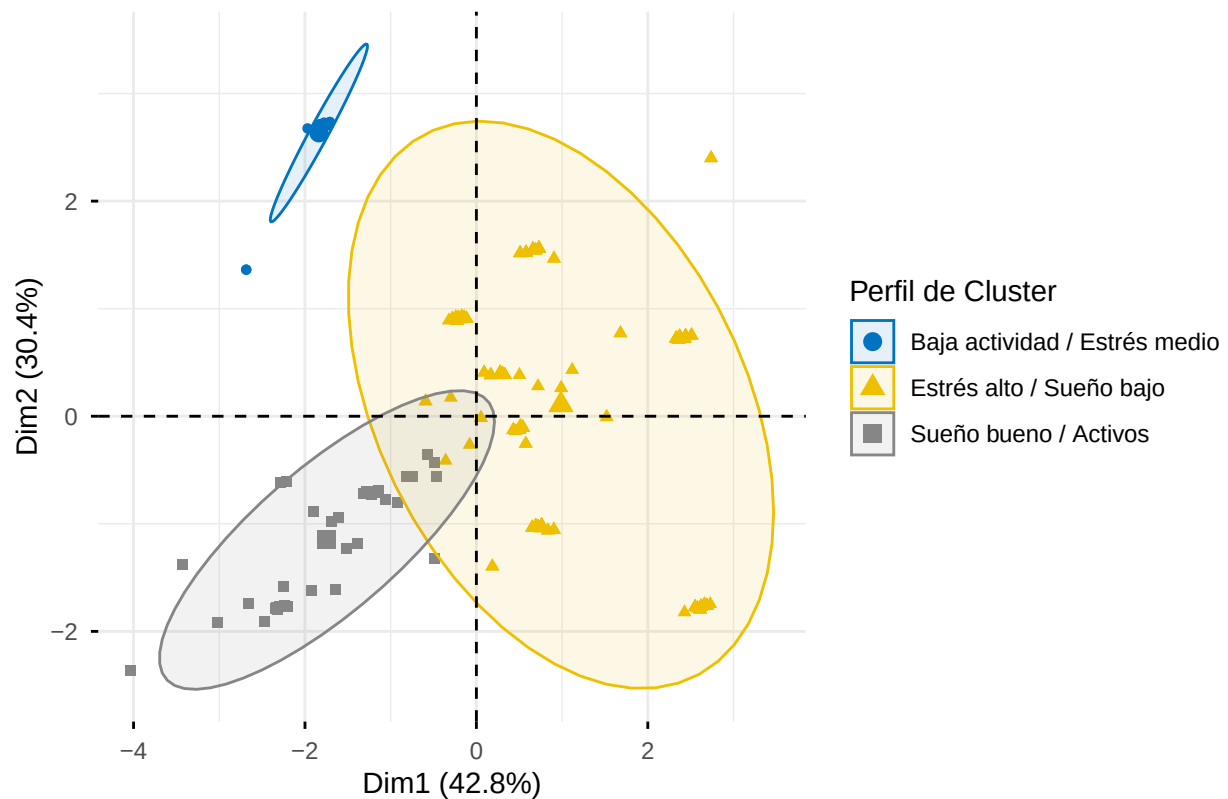
df_1 <- df_1 %>%
  mutate(Cluster = case_when(
    Cluster == 1 ~ "Estrés alto / Sueño bajo",
    Cluster == 2 ~ "Sueño bueno / Activos",
    Cluster == 3 ~ "Baja actividad / Estrés medio",
    TRUE ~ as.character(Cluster)
  ))

# Hacemos PCA con las mismas variables estandarizadas
pca_res <- prcomp(df_1_scaled, center = TRUE, scale. = TRUE)

# PCA con puntos coloreados y leyenda con nombres descriptivos
fviz_pca_ind(pca_res,
  geom.ind = "point",
  col.ind = df_1$Cluster,          # Usa nombres descriptivos
  palette = "jco",
  addEllipses = TRUE,
  legend.title = "Perfil de Cluster") +
  ggtitle("PCA - Individuos agrupados por perfil de Cluster") +
  theme(plot.title = element_text(hjust = 0.5))
```

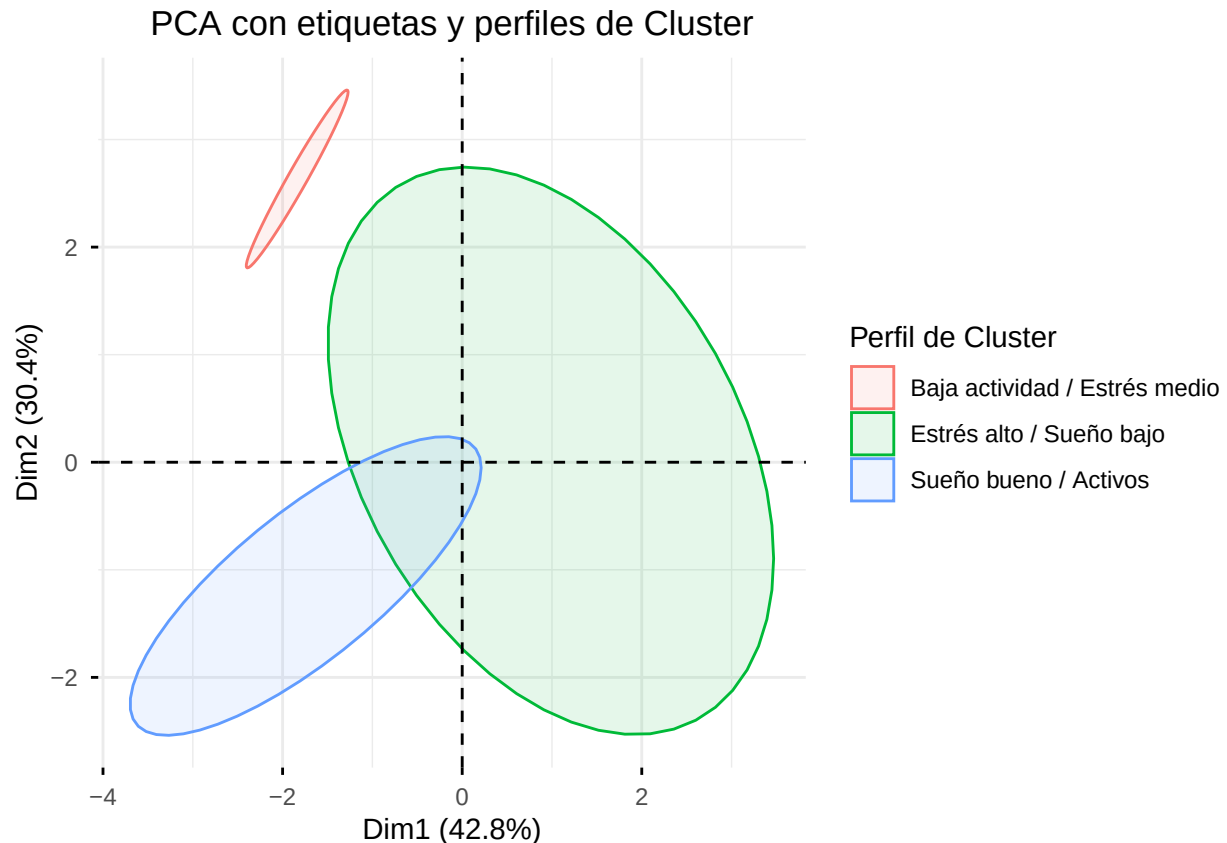
```
## Ignoring unknown labels:
## * linetype : "Perfil de Cluster"
```

PCA – Individuos agrupados por perfil de Cluster



```
# Hacemos PCA con etiquetas de Person.ID + nombres descriptivos en leyenda
fviz_pca_ind(pca_res,
  geom.ind = "text",
  col.ind = df_1$Cluster,
  label = df_1$Person.ID,
  repel = TRUE,
  addEllipses = TRUE,
  legend.title = "Perfil de Cluster") +
ggtitle("PCA con etiquetas y perfiles de Cluster") +
theme(plot.title = element_text(hjust = 0.5))
```

```
## Ignoring unknown labels:
## * linetype : "Perfil de Cluster"
## * shape : "Perfil de Cluster"
```



Mediante K-Means identificamos 3 grupos diferenciados. Los grupos obtenidos son: -Estrés alto / Sueño bajo -Sueño bueno / Activos -Baja actividad / Estrés medio

El PCA muestra que estos clústeres están bien separados, explicando alrededor del 75% de la variabilidad total en las dos primeras dimensiones.

Los individuos con mayor actividad física y mejor sueño se agrupan de forma consistente y opuesta a quienes presentan alto estrés y baja calidad de sueño.

Podemos concluir que existen patrones claros entre sueño, actividad física y estrés, útiles para segmentar perfiles de salud y orientar intervenciones personalizadas.

#justificar porque k=3

Conclusiones Después de analizar todos los datos del proyecto, hemos visto que el sueño no depende solo de cuántas horas dormimos, sino de muchos factores del día a día como el estrés, el ejercicio o incluso el peso de cada persona. Aquí dejamos lo más importante que hemos aprendido:

Estrés, sueño y actividad física están muy conectados

- Cuanto más estrés tiene una persona, peor duerme. La relación es muy fuerte (correlación de -0.89), así que no es casualidad.
 - También vimos que las personas que hacen más actividad física suelen dormir mejor y tener menos estrés. El ejercicio ayuda mucho a desconectar.
-

El peso (IMC) también influye en la calidad del sueño

- La gente con sobrepeso u obesidad suele tener peor calidad del sueño que las personas con un peso normal.
 - El cambio más importante ocurre al pasar de peso normal a sobrepeso. Entre sobrepeso y obesidad no cambia tanto, lo más crítico es ese primer salto.
-

¿Qué variables afectan más a dormir bien? Según nuestros modelos estadísticos y la regresión:

- La variable que más influye es el **nivel de estrés**.
 - Luego va la **duración del sueño** (cuántas horas dormimos realmente).
 - Cosas como los pasos diarios, el pulso o la edad influyen menos.
 - O sea, no es solo dormir 8 horas, sino cómo llegamos mentalmente al final del día.
-

Los trastornos del sueño sí afectan mucho

- Las personas con insomnio o apnea duermen notablemente peor que las demás.
 - Además, hicimos un modelo para predecir si alguien tiene un trastorno del sueño y salió bastante bien: $AUC = 0.88$, sensibilidad del 93%. Esto significa que los datos sirven realmente para detectar estos problemas.
-

Existen distintos perfiles de personas según sus hábitos El análisis de clústeres nos mostró 3 grupos claros:

1. Personas con **alto estrés y mal sueño**.
2. Personas con **buen sueño, hacen ejercicio y llevan hábitos saludables**.
3. Personas con **poca actividad física y estrés medio**.

La variable que más separa los grupos es la **calidad del sueño**, más que la edad o el género.

Conclusión final En resumen:

- Dormir bien no es solo cuestión de tiempo, sino de hábitos. Los hábitos diarios son clave.
 - El **estrés es uno de los factores que empeora más el sueño**.
 - Hacer ejercicio y mantener un peso saludable ayuda mucho a descansar mejor.
 - Este estudio no solo confirma cosas que ya se intuían, sino que además lo hace con datos reales y modelos que funcionan bastante bien.
-

Parte del código de este trabajo ha sido generado con herramientas de Inteligencia Artificial.